

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh đề tài

Trong các hệ thống nhúng hiện đại, nhiều tác vụ thường phải hoạt động đồng thời để xử lý cảm biến, điều khiển thiết bị, truyền thông dữ liệu, ghi log và phản ứng với các sự kiện khẩn cấp. Khác với phần mềm ứng dụng thông thường, hệ thống nhúng thường bị giới hạn về tài nguyên xử lý, bộ nhớ và đặc biệt phải đáp ứng các yêu cầu thời gian thực. Một tác vụ không chỉ cần cho ra kết quả đúng, mà còn phải hoàn thành trong khoảng thời gian cho phép.

Ví dụ, trong hệ thống báo cháy thông minh, tác vụ phát hiện cháy và kích hoạt cảnh báo phải được xử lý trong thời gian rất ngắn. Nếu tác vụ này bị trì hoãn bởi các tác vụ ít quan trọng hơn như ghi log hoặc gửi dữ liệu định kỳ, hệ thống có thể gây ra hậu quả nghiêm trọng. Tương tự, trong nhà máy có Emergency Stop, khi có tín hiệu dừng khẩn cấp, tác vụ điều khiển an toàn phải được ưu tiên xử lý ngay lập tức.

Vì vậy, việc kiểm thử hệ thống nhúng đa tiến trình thời gian thực không chỉ dừng lại ở kiểm thử chức năng, mà còn phải kiểm thử các đặc tính phi chức năng như thời gian phản hồi, độ trễ, khả năng đáp ứng deadline, khả năng lập lịch, cơ chế ưu tiên, preemption và đồng bộ hóa tài nguyên dùng chung.

Đề tài “Xây dựng và kiểm thử hệ nhúng đa tiến trình thời gian thực” được thực hiện nhằm xây dựng một môi trường mô phỏng hệ thống nhúng sử dụng FreeRTOS Simulator, từ đó thiết kế và thực hiện các test case để đánh giá hành vi của hệ thống trong các tình huống thực tế.

1.2. Lý do chọn đề tài

FreeRTOS là một hệ điều hành thời gian thực nhẹ, được sử dụng phổ biến trong các hệ thống nhúng. FreeRTOS cung cấp các cơ chế quan trọng như task scheduling, priority scheduling, preemption, semaphore, mutex, queue và delay theo tick hệ thống. Đây là nền tảng phù hợp để mô phỏng và kiểm thử hệ thống nhúng đa tiến trình.

Đề tài được lựa chọn vì các lý do sau:

Thứ nhất, đề tài giúp sinh viên hiểu rõ bản chất của hệ thống thời gian thực. Trong hệ thống thời gian thực, kết quả đúng nhưng đến muộn vẫn có thể bị xem là sai. Do đó, việc đo thời gian phản hồi, deadline miss, latency và jitter là yêu cầu quan trọng.

Thứ hai, đề tài giúp kiểm chứng cơ chế lập lịch của RTOS. Thông qua các test case như “Camera An Ninh Phát Hiện Xâm Nhập” hoặc “Nhà Máy Có Emergency Stop”, nhóm có thể đánh giá task ưu tiên cao có được scheduler cấp CPU đúng lúc hay không.

Thứ ba, đề tài giúp làm rõ vấn đề đồng thời và đồng bộ hóa trong hệ thống đa task. Khi nhiều task cùng truy cập tài nguyên dùng chung, hệ thống có thể xuất hiện race condition, deadlock hoặc priority inversion. Các lỗi này khó phát hiện nếu chỉ kiểm thử chức năng thông thường.

Thứ tư, đề tài tạo cơ hội áp dụng kỹ thuật kiểm thử phần mềm vào hệ thống nhúng. Thay vì chỉ viết chương trình chạy được, nhóm cần thiết kế test case, thu thập log, phân tích kết quả và đưa ra kết luận pass/fail dựa trên dữ liệu đo đạc.

1.3. Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống mô phỏng đa tiến trình thời gian thực sử dụng FreeRTOS Simulator và thực hiện kiểm thử các cơ chế lập lịch, ưu tiên, preemption, deadline, đồng bộ hóa và xử lý lỗi.

Mục tiêu	Mô tả chi tiết	Ví dụ kịch bản
Xây dựng hệ thống FreeRTOS đa task	Nhiều task chạy đồng thời, có priority, period, deadline	Sensor_Task, Control_Task, Logger_Task, Emergency_Task
Kiểm thử scheduler & priority	Đánh giá task priority cao chạy trước task thấp	Camera An Ninh Phát Hiện Xâm Nhập
Phân tích deadline	Hard/Soft deadline, đo response time, latency, jitter	Hệ Thống Báo Cháy Thông Minh, Drone Tránh Chướng Ngại Vật
Kiểm thử concurrency & synchronization	Race condition, deadlock, priority inversion	Hệ Thống Ghi Log Nhà Thông Minh, Robot Công Nghiệp Tránh Va Chạm
Fault Injection	Tạo lỗi CPU overload, task timeout, stack overflow	Nhà Máy Có Emergency Stop, Faulty Sensor
Thu thập log & metrics	Execution time, response time, CPU usage, context switch	Tất cả test case
Pass/Fail evaluation	So sánh expected vs actual	Mọi test case thực tế

1.4. Phạm vi thực hiện

Phạm vi đề tài tập trung vào kiểm thử hệ thống nhúng đa tiến trình thời gian thực trong môi trường mô phỏng FreeRTOS trên máy tính cá nhân. Đề tài không tập trung xây dựng web application, AI application hoặc hệ thống quản lý dữ liệu phức tạp. Các công cụ phụ trợ như script phân tích log hoặc biểu đồ chỉ đóng vai trò hỗ trợ cho quá trình kiểm thử.

Phạm vi kỹ thuật bao gồm:

- FreeRTOS Simulator.
- Ngôn ngữ lập trình C.
- Môi trường Windows hoặc Linux.
- VSCode hoặc Visual Studio.
- Các task mô phỏng hệ thống nhúng.
- Mutex, semaphore, queue.
- Logger ghi nhận timestamp và trạng thái task.
- Deadline monitor để phát hiện deadline miss.
- Fault injector để tạo lỗi có kiểm soát.
- File log hoặc CSV để phân tích kết quả.

Các nội dung không thuộc trọng tâm đề tài:

- Không xây dựng hệ thống web quản lý quy mô lớn.
- Không phát triển ứng dụng AI.
- Không xây dựng dashboard phức tạp nếu chưa hoàn thiện phần kiểm thử.
- Không tập trung vào giao diện người dùng.

1.5. Phương pháp thực hiện

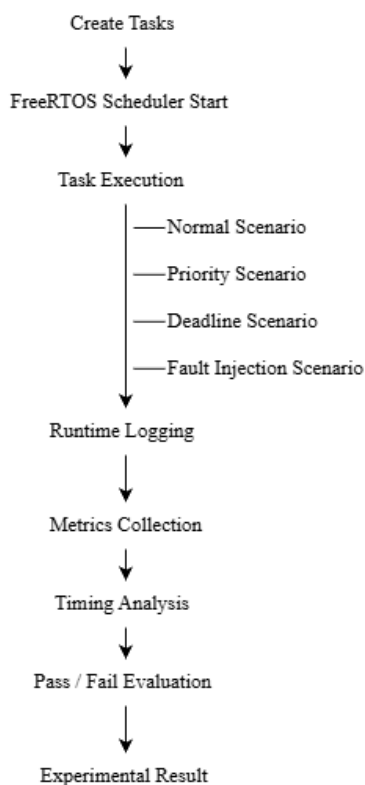
Đề tài được thực hiện theo phương pháp thực nghiệm kết hợp phân tích học thuật nhằm đánh giá hành vi của hệ thống nhúng đa tiến trình thời gian thực trong môi trường mô phỏng FreeRTOS.

Trước hết, nhóm nghiên cứu cơ sở lý thuyết liên quan đến hệ thống nhúng, RTOS, FreeRTOS, scheduler, priority scheduling, priority preemption, concurrency, synchronization và deadline analysis. Đây là nền tảng quan trọng để thiết kế kiến trúc hệ thống, xây dựng task và xác định các tiêu chí kiểm thử phù hợp.

Tiếp theo, nhóm xây dựng hệ thống FreeRTOS Simulator gồm nhiều task có mức priority, period, WCET và deadline khác nhau. Các task được thiết kế nhằm mô phỏng các thành phần thường gặp trong hệ thống nhúng thực tế như cảm biến, điều khiển, truyền thông, ghi log và xử lý sự kiện khẩn cấp.

Sau giai đoạn xây dựng hệ thống, nhóm tiến hành thiết kế bộ test case theo các nhóm kiểm thử chính gồm scheduler testing, priority scheduling, priority preemption, hard deadline, soft deadline, synchronization, concurrency, fault injection và timing analysis. Mỗi test case đều được xây dựng với mục tiêu rõ ràng, bao gồm lý do thực hiện, giá trị kiểm thử, phương pháp đo đạc, kết quả mong đợi và tiêu chí đánh giá pass/fail.

Quy trình thực hiện kiểm thử của đề tài được mô tả trong Hình 1.1.



Hình 1.1. Quy trình thực hiện kiểm thử hệ thống nhúng đa tiến trình thời gian thực.

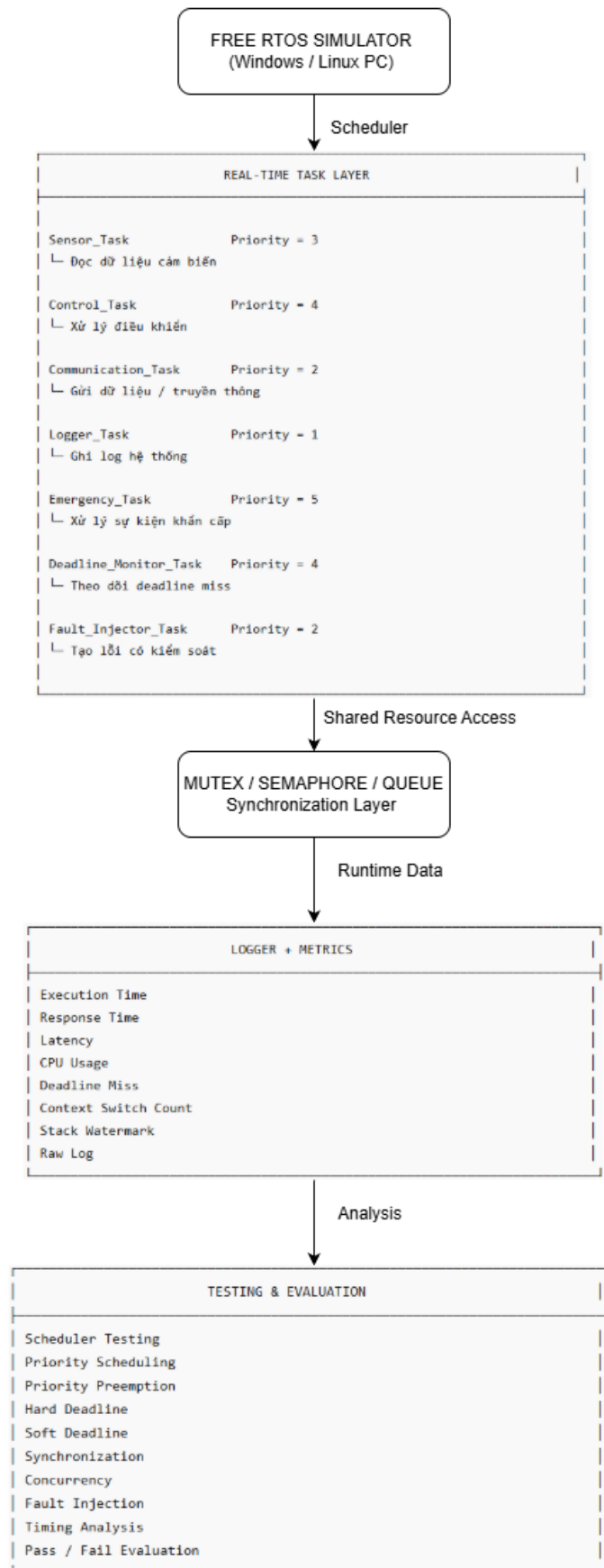
Trong quá trình chạy kiểm thử, hệ thống ghi nhận dữ liệu runtime thông qua cơ chế logging theo timestamp. Các metrics được thu thập bao gồm execution time, response time, latency, jitter, deadline miss, context switch count, CPU usage và stack watermark. Dữ liệu thu được được sử dụng để xây dựng bảng kết quả, biểu đồ phân tích và đánh giá hiệu năng hệ thống.

Cuối cùng, nhóm tiến hành phân tích kết quả thực nghiệm dựa trên dữ liệu đo đạc thực tế nhằm đánh giá tính đúng đắn của scheduler, cơ chế ưu tiên, khả năng đáp ứng deadline, hiệu quả đồng bộ hóa và khả năng xử lý lỗi của hệ thống. Từ đó, nhóm đưa ra kết luận, chỉ ra hạn chế và đề xuất hướng phát triển trong tương lai.

1.6. Đối tượng kiểm thử

Đối tượng kiểm thử trong đề tài là hệ thống FreeRTOS mô phỏng nhiều task chạy đồng thời. Hệ thống sử dụng FreeRTOS Simulator làm môi trường thực nghiệm nhằm kiểm thử các cơ chế scheduler, priority scheduling, priority preemption, synchronization, deadline handling và fault management trong hệ thống nhúng đa tiến trình thời gian thực.

Kiến trúc tổng quan của hệ thống kiểm thử được trình bày trong Hình 1.2.



Hình 1.2. Kiến trúc tổng quan hệ thống kiểm thử FreeRTOS Simulator.

Hệ thống bao gồm lớp task thời gian thực, lớp đồng bộ hóa tài nguyên, lớp ghi nhận dữ liệu runtime và lớp đánh giá kết quả kiểm thử. Các task chính của hệ thống được mô tả trong Bảng 1.2.

Task	Vai trò	Đặc điểm kiểm thử
Sensor_Task	Mô phỏng đọc dữ liệu cảm biến	Kiểm thử chu kỳ, jitter, queue
Control_Task	Xử lý điều khiển	Kiểm thử deadline và priority
Communication_Task	Gửi dữ liệu ra ngoài	Kiểm thử soft deadline và blocking
Logger_Task	Ghi log hệ thống	Kiểm thử shared resource và mutex
Emergency_Task	Xử lý sự kiện khẩn cấp	Kiểm thử priority preemption và hard deadline
Deadline_Monitor_Task	Theo dõi deadline	Kiểm thử deadline miss
Fault_Injector_Task	Tạo lỗi có kiểm soát	Kiểm thử fault injection

Việc thiết kế các task này giúp hệ thống có đủ điều kiện để kiểm thử các đặc trưng quan trọng của hệ nhúng thời gian thực như scheduler behavior, priority preemption, deadline miss, shared resource conflict và fault injection.

1.7. Các nhóm kiểm thử chính

Bộ kiểm thử của đề tài được chia thành các nhóm chính sau:

Nhóm kiểm thử	Mục tiêu	Test case điển hình
Scheduler Testing	Kiểm tra task được lập lịch đúng	Sensor Kiểm Soát Người Ra Vào, Trạm Quan Trắc Thời Tiết
Priority Scheduling	Task priority cao chạy trước	Camera An Ninh Phát Hiện Xâm Nhập

Priority Preemption	Task khẩn cấp chiếm CPU ngay	Nhà Máy Có Emergency Stop
Hard Deadline	Task quan trọng hoàn thành đúng thời hạn	Hệ Thống Báo Cháy Thông Minh
Soft Deadline	Task ít quan trọng hoàn thành chậm vẫn chấp nhận	Hệ Thống Giám Sát Chất Lượng Không Khí
Synchronization	Bảo vệ tài nguyên dùng chung	Hệ Thống Ghi Log Nhà Thông Minh
Concurrency	Phát hiện race condition, deadlock	Robot Công Nghiệp Tránh Va Chạm
Fault Injection	Kiểm tra khả năng phát hiện lỗi	Faulty Sensor, CPU Overload
Timing Analysis	Đo latency, jitter, response time	Drone Tránh Chướng Ngại Vật
Pass/Fail Evaluation	Đánh giá kết quả	Tất cả test case

1.8. Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, đề tài giúp làm rõ mối quan hệ giữa lý thuyết hệ điều hành thời gian thực và kiểm thử phần mềm. Các khái niệm như priority scheduling, preemption, context switching, mutex, semaphore, hard deadline và soft deadline không chỉ được trình bày ở mức lý thuyết, mà còn được kiểm chứng thông qua dữ liệu thực nghiệm.

Về mặt thực tiễn, đề tài mô phỏng các tình huống có thể xuất hiện trong hệ thống nhúng thực tế. Ví dụ, trong “Hệ Thống Báo Cháy Thông Minh”, deadline miss có thể dẫn đến cảnh báo chậm. Trong “Drone Tránh Chướng Ngại Vật”, response time cao có thể khiến hệ thống phản ứng không kịp. Trong “Hệ Thống Ghi Log Nhà Thông Minh”, việc nhiều task cùng ghi log có thể gây race condition nếu không có mutex.

Do đó, kết quả của đề tài không chỉ chứng minh hệ thống chạy được, mà còn chứng minh nhóm có khả năng thiết kế kiểm thử, đo đạc, phân tích và đánh giá hành vi của hệ thống nhúng thời gian thực.

CHƯƠNG 2 – CƠ SỞ LÝ THUYẾT

Chương này trình bày các kiến thức nền tảng phục vụ trực tiếp cho việc xây dựng và kiểm thử hệ thống nhúng đa tiến trình thời gian thực sử dụng FreeRTOS Simulator. Nội dung chương tập trung vào các khái niệm về Embedded Systems, RTOS, FreeRTOS, scheduler, priority scheduling, synchronization, deadline handling và timing analysis. Các nội dung này được liên kết trực tiếp với hệ thống, task và test case đã giới thiệu trong Chương 1 nhằm tạo cơ sở cho quá trình thiết kế, cài đặt, kiểm thử và đánh giá thực nghiệm trong các chương tiếp theo.

2.1 Embedded Systems

Trong các hệ thống kỹ thuật hiện đại, nhiều thiết bị không còn chỉ thực hiện một chức năng đơn lẻ mà phải xử lý đồng thời nhiều nhiệm vụ với yêu cầu phản hồi nghiêm ngặt về thời gian. Những hệ thống này thường được triển khai dưới dạng hệ thống nhúng, nơi khả năng tính toán, bộ nhớ và năng lượng đều bị giới hạn nhưng vẫn phải đảm bảo tính đúng hạn trong quá trình vận hành.

Hệ thống nhúng là các hệ thống máy tính chuyên dụng được tích hợp vào thiết bị hoặc sản phẩm cụ thể nhằm thực hiện các chức năng xác định trong môi trường vận hành thực tế. Khác với máy tính đa năng, hệ thống nhúng vận hành trong các điều kiện giới hạn và thường yêu cầu độ tin cậy, tính ổn định cũng như khả năng đáp ứng thời gian thực ở mức cao.

Trong các hệ thống nhúng hiện đại, nhiều tác vụ phải hoạt động đồng thời để thực hiện các chức năng như thu thập dữ liệu cảm biến, xử lý tín hiệu điều khiển, truyền thông dữ liệu, ghi nhận trạng thái hệ thống và phản hồi các sự kiện bất thường. Khi số lượng tác vụ gia tăng cùng với sự khác biệt về mức độ ưu tiên và yêu cầu thời gian xử lý, mô hình thực thi tuần tự truyền thống không còn đảm bảo hiệu năng vận hành. Điều này dẫn đến nhu cầu sử dụng các cơ chế quản lý đa nhiệm, lập lịch thực thi và điều phối tài nguyên trong môi trường thời gian thực.

Để giải quyết bài toán đa nhiệm và kiểm soát thời gian đáp ứng trong hệ thống nhúng, các hệ điều hành thời gian thực (RTOS) thường được sử dụng như một lớp trung gian chịu trách nhiệm quản lý scheduler, task execution, synchronization và timing control. Đây cũng là định hướng kỹ thuật được sử dụng trong proposal của đề tài nhằm xây dựng môi trường kiểm thử hệ thống nhúng đa tiến trình thời gian thực.

Như đã trình bày trong Chương 1, các kịch bản ứng dụng được lựa chọn cho đồ án, bao gồm **Drone Tránh Chướng Ngại Vật**, **Camera An Ninh Phát Hiện Xâm Nhập**, **Sensor Kiểm Soát Người Ra Vào**, **Hệ Thống Báo Cháy Thông Minh** và **Trạm Quan**

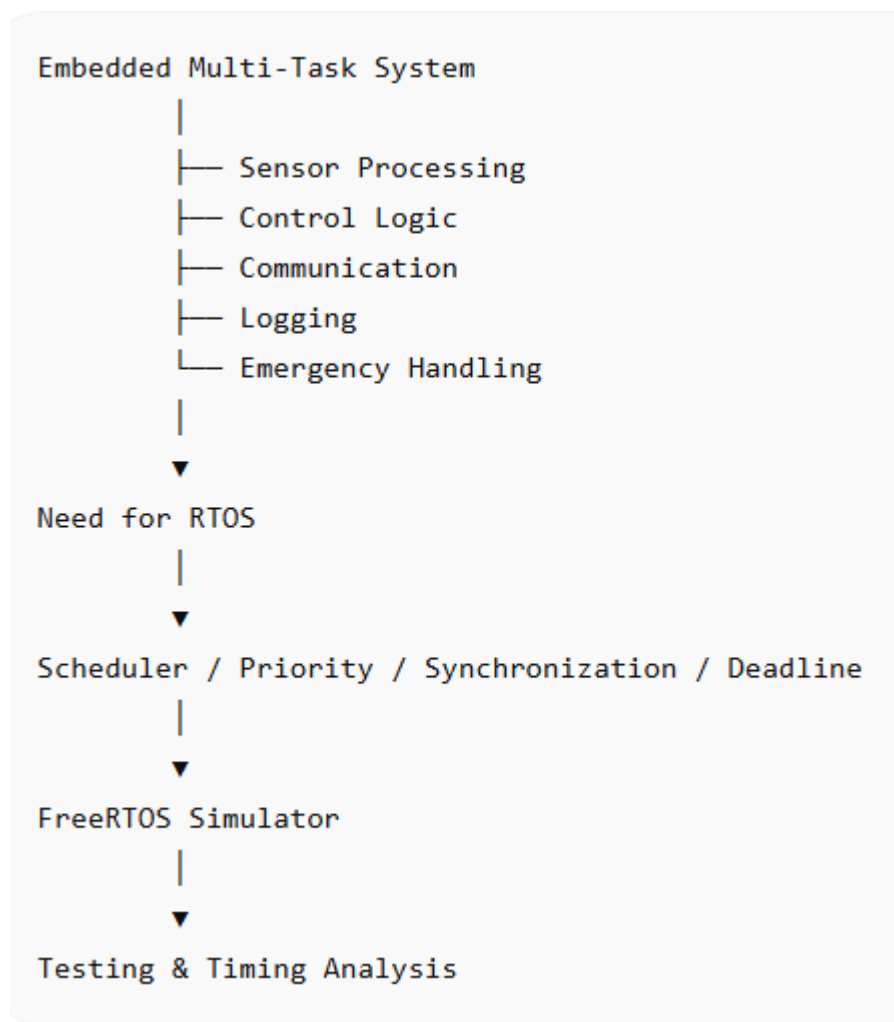
Trắc Thời Tiết, đều yêu cầu xử lý đồng thời nhiều tác vụ với các ràng buộc thời gian khác nhau. Trong các hệ thống này, dữ liệu cảm biến phải được cập nhật liên tục, tác vụ điều khiển phải phản hồi trong giới hạn thời gian xác định, trong khi các nhiệm vụ phụ trợ như truyền thông hoặc ghi log vẫn cần duy trì hoạt động song song. Sự đồng thời của nhiều tác vụ tạo ra bài toán về scheduler, priority scheduling, concurrency, synchronization và deadline handling.

Trong phạm vi đồ án, nhóm sử dụng **FreeRTOS Simulator** để tái tạo môi trường vận hành của hệ thống nhúng đa tiến trình thời gian thực. Các task triển khai bao gồm **Sensor_Task**, **Control_Task**, **Communication_Task**, **Logger_Task**, **Emergency_Task**, **Deadline_Monitor_Task** và **Fault_Injector_Task**. Mỗi task được cấu hình với mức priority, chu kỳ hoạt động và deadline khác nhau nhằm phục vụ việc đánh giá cơ chế scheduler, priority preemption, synchronization và timing analysis trong môi trường kiểm thử.

Thông qua môi trường mô phỏng này, đề tài có thể quan sát trực tiếp hành vi của scheduler, khả năng preemption của các task ưu tiên cao, hiệu quả của cơ chế đồng bộ tài nguyên và mức độ đáp ứng deadline của từng nhiệm vụ. Dữ liệu runtime thu thập từ các task tiếp tục được sử dụng cho timing analysis nhằm đánh giá response time, latency, context switching và tính đúng hạn của hệ thống. Các kết quả này đóng vai trò quan trọng trong quá trình thiết kế test case, phân tích pass/fail và đánh giá hiệu năng của hệ thống FreeRTOS Simulator.

Bên cạnh bài toán lập lịch, việc nhiều task cùng truy cập tài nguyên dùng chung cũng đặt ra yêu cầu về synchronization và concurrency control. Nếu không có chiến lược quản lý thích hợp, hệ thống có thể xuất hiện hiện tượng race condition, deadlock hoặc priority inversion, làm suy giảm tính ổn định và khả năng đáp ứng thời gian thực của hệ thống. Những vấn đề này tạo nền tảng lý thuyết cho các mục tiếp theo về cơ chế RTOS, FreeRTOS, concurrency, và test case evaluation.

Gợi ý hình minh họa:
Hình 2.1. Mối liên hệ giữa hệ thống nhúng đa nhiệm và môi trường kiểm thử FreeRTOS Simulator



2.2 RTOS và FreeRTOS

Như đã trình bày trong Chương 1, các kịch bản kiểm thử của đề tài đều đặt ra yêu cầu xử lý đồng thời nhiều tác vụ với các ràng buộc thời gian khác nhau. Trong các hệ thống nhúng đa nhiệm, việc nhiều task cùng tồn tại với mức độ ưu tiên và yêu cầu thời gian thực riêng biệt đòi hỏi một cơ chế quản lý thực thi có khả năng điều phối tài nguyên xử lý theo cách có dự đoán. Đây là cơ sở dẫn tới việc sử dụng hệ điều hành thời gian thực (RTOS).

RTOS đóng vai trò như một lớp quản lý trung gian, chịu trách nhiệm phân bổ CPU, quản lý trạng thái task và điều phối quá trình chuyển đổi thực thi giữa các nhiệm vụ đang hoạt động. Các trạng thái như Ready, Running, Blocked và Suspended cho phép hệ thống kiểm soát vòng đời của task, trong khi cơ chế lập lịch đảm bảo tài nguyên xử lý được cấp phát phù hợp với mức priority và deadline của từng nhiệm vụ.

Trong môi trường FreeRTOS Simulator của nhóm, các cơ chế này được tái tạo nhằm phản ánh hành vi vận hành của hệ thống nhúng đa tiến trình thời gian thực. Các task chức năng, task giám sát deadline, task logging và task xử lý sự kiện khẩn cấp được quản lý

thông qua scheduler với mức priority, chu kỳ hoạt động và deadline riêng biệt. Điều này cho phép nhóm quan sát trực tiếp cơ chế priority scheduling và priority preemption trong các test case như **Camera An Ninh Phát Hiện Xâm Nhập** và **Nhà Máy Có Emergency Stop**, nơi các task ưu tiên cao được cấp CPU ngay khi sẵn sàng thực thi.

Trong số các RTOS hiện nay, FreeRTOS được sử dụng rộng rãi trong các hệ thống nhúng nhờ kiến trúc gọn nhẹ, khả năng cấu hình linh hoạt và hỗ trợ đầy đủ các cơ chế quản lý task, synchronization và inter-task communication. Các API do FreeRTOS cung cấp cho phép xây dựng môi trường thực thi đa nhiệm có kiểm soát, phù hợp với yêu cầu kiểm thử scheduler, deadline handling, synchronization và concurrency của đề tài.

Sử dụng FreeRTOS Windows PC Simulator, nhóm triển khai hệ thống kiểm thử mà không phụ thuộc trực tiếp vào phần cứng thực. Môi trường mô phỏng này cho phép thu thập log runtime để đánh giá execution time, response time, latency, deadline miss và context switch count, đồng thời hỗ trợ kiểm thử các cơ chế đồng bộ hóa và điều phối tài nguyên trong nhiều tình huống vận hành khác nhau.

Thông qua việc triển khai FreeRTOS Simulator, nhóm xác định rõ mối quan hệ giữa khái niệm RTOS, cơ chế scheduler, priority scheduling, priority preemption và các kịch bản kiểm thử thực tế. Từ góc độ đề tài, RTOS và FreeRTOS không chỉ đóng vai trò môi trường thực thi mà còn là nền tảng để khảo sát hành vi của scheduler trong hệ thống đa tiến trình thời gian thực. Những nội dung này sẽ được phân tích chi tiết hơn trong các mục tiếp theo về scheduler, concurrency, synchronization và timing analysis.

2.3 Scheduler, Priority Scheduling và Priority Preemption

Sau khi thiết lập nền tảng về RTOS và FreeRTOS ở mục trước, cơ chế lập lịch trở thành thành phần trung tâm quyết định cách các task được thực thi trong hệ thống đa tiến trình thời gian thực. Trong môi trường RTOS, scheduler không chỉ đơn thuần lựa chọn task tiếp theo để cấp CPU mà còn đóng vai trò điều phối tài nguyên xử lý theo mức độ ưu tiên, trạng thái task và các ràng buộc thời gian của hệ thống.

Scheduler là cơ chế chịu trách nhiệm quản lý quá trình thực thi của các task dựa trên các trạng thái như Ready, Running, Blocked và Suspended. Trong các hệ thống nhúng đa nhiệm, nhiều task có thể cùng tồn tại và cạnh tranh tài nguyên xử lý tại cùng một thời điểm. Vì vậy, hiệu quả của scheduler ảnh hưởng trực tiếp đến khả năng đáp ứng deadline, độ trễ phản hồi và tính ổn định của toàn bộ hệ thống.

Trong FreeRTOS, cơ chế lập lịch được xây dựng chủ yếu dựa trên **priority scheduling**. Theo mô hình này, các task có mức priority cao hơn sẽ được ưu tiên thực thi trước các task priority thấp hơn khi cùng ở trạng thái Ready. Điều này đặc biệt quan trọng

đối với các hệ thống thời gian thực, nơi mức độ quan trọng của từng nhiệm vụ không đồng nhất và việc phân bổ CPU cần phản ánh đúng mức ưu tiên vận hành của hệ thống.

Liên hệ với định hướng đã trình bày trong Chương 1, bài toán scheduler được thể hiện rõ trong các kịch bản như **Camera An Ninh Phát Hiện Xâm Nhập**, **Drone Tránh Chướng Ngại Vật** và **Nhà Máy Có Emergency Stop**. Trong các hệ thống này, task xử lý cảnh báo hoặc điều khiển khẩn cấp phải được ưu tiên cao hơn các nhiệm vụ phụ trợ như ghi log, truyền thông hoặc cập nhật trạng thái định kỳ. Nếu cơ chế lập lịch không phản ánh đúng thứ tự ưu tiên, hệ thống có thể xuất hiện hiện tượng phản hồi chậm, deadline miss hoặc suy giảm khả năng vận hành trong các tình huống khẩn cấp.

Trong môi trường **FreeRTOS Simulator** của đề tài, cơ chế này được triển khai thông qua các task có mức priority khác nhau như **Sensor_Task**, **Control_Task**, **Communication_Task**, **Logger_Task** và **Emergency_Task**. Các task này được cấu hình với chu kỳ thực thi, thời gian xử lý và deadline riêng nhằm mô phỏng các thành phần thường gặp trong hệ thống nhúng thực tế. Việc gán priority khác nhau cho từng task cho phép quan sát trực tiếp cách scheduler điều phối CPU trong nhiều điều kiện vận hành khác nhau.

Gắn liền với priority scheduling là cơ chế **priority preemption**. Trong hệ thống preemptive scheduling, khi một task có priority cao chuyển sang trạng thái Ready, scheduler có thể tạm dừng task đang chạy để cấp CPU cho task ưu tiên cao hơn. Đây là cơ chế đặc biệt quan trọng trong hệ thống thời gian thực, bởi nhiều nhiệm vụ không thể chờ đến khi task hiện tại hoàn thành mới được xử lý.

Điều này được thể hiện rõ trong test case **Nhà Máy Có Emergency Stop**, nơi **Emergency_Task** đại diện cho tác vụ dừng khẩn cấp của hệ thống. Khi tín hiệu emergency xuất hiện, task này phải được scheduler cấp CPU gần như tức thời, bất kể các task khác đang thực thi. Tương tự, trong kịch bản **Camera An Ninh Phát Hiện Xâm Nhập**, tác vụ phát hiện sự kiện bất thường cần ưu tiên cao hơn quá trình ghi log hoặc truyền dữ liệu định kỳ. Các tình huống này được sử dụng trong đồ án để kiểm thử trực tiếp cơ chế priority scheduling và priority preemption của FreeRTOS.

Thông qua FreeRTOS Simulator, nhóm có thể ghi nhận dữ liệu runtime nhằm đánh giá hành vi của scheduler dưới nhiều điều kiện vận hành khác nhau. Các chỉ số như **response time**, **latency**, **context switch count** và **deadline miss** được sử dụng để phân tích khả năng phản ứng của scheduler đối với các task có mức priority khác nhau. Những dữ liệu này không chỉ phục vụ việc đánh giá pass/fail của test case mà còn tạo nền tảng cho quá trình timing analysis ở các mục tiếp theo.

Từ góc độ đề tài, scheduler, priority scheduling và priority preemption không được xem như các khái niệm lý thuyết độc lập mà được sử dụng như công cụ để khảo sát hành

vi của hệ thống nhúng đa tiến trình thời gian thực. Việc hiểu rõ cơ chế lập lịch của FreeRTOS là điều kiện cần để triển khai task design, synchronization strategy, deadline analysis và đánh giá hiệu năng của hệ thống kiểm thử.

Gợi ý hình nên chèn

Hình 2.3. Scheduler và cơ chế Priority Preemption trong FreeRTOS Simulator

Time →

Logger_Task (Priority 1)



Control_Task (Priority 4)



Emergency_Task (Priority 5)



Preemption:

Emergency_Task becomes READY

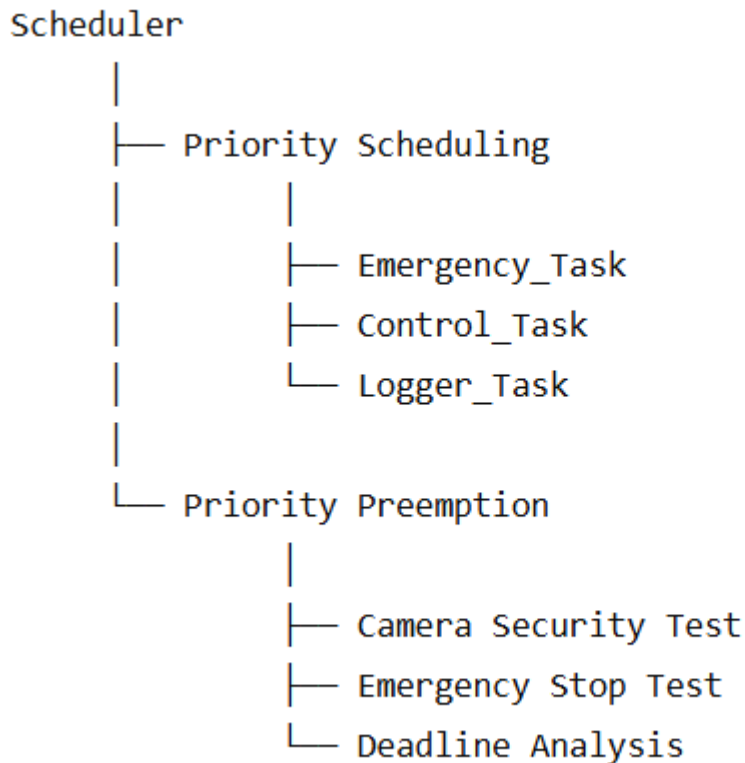


Scheduler interrupts lower priority task



CPU reassigned immediately

Hình 2.4. Quan hệ giữa Scheduler, Priority Scheduling và Test Case của đề tài



2.4 Concurrency và Synchronization

Tiếp nối nội dung về scheduler và priority preemption, các vấn đề về concurrency và synchronization trở thành trọng tâm trong việc đảm bảo tính ổn định của hệ thống nhúng đa tiến trình. Khi nhiều task cùng truy cập tài nguyên dùng chung như biến hệ thống, queue hoặc thiết bị ngoại vi, thứ tự thực thi không kiểm soát có thể gây ra các hiện tượng như race condition, deadlock hoặc priority inversion. Những vấn đề này được nhóm kiểm chứng trực tiếp thông qua FreeRTOS Simulator và các test case thực tế.

Trong FreeRTOS, việc kiểm soát concurrency được triển khai thông qua các cơ chế như mutex, binary semaphore, counting semaphore và queue. Các cơ chế này cho phép điều phối truy cập tài nguyên dùng chung, đồng bộ hóa sự kiện giữa các task và hỗ trợ trao đổi dữ liệu trong môi trường đa nhiệm. Nhờ đó, hệ thống có thể duy trì tính nhất quán của dữ liệu và hạn chế các lỗi phát sinh do nhiều task cùng hoạt động đồng thời.

Trong đồ án, các task như Sensor_Task, Control_Task, Logger_Task và Emergency_Task đôi khi phải truy cập đồng thời cùng một biến hoặc queue. Việc quản lý truy cập này được thực hiện thông qua các cơ chế mutex, semaphore và queue. FreeRTOS cung cấp API để tạo và quản lý các cơ chế đồng bộ này, cho phép task thực hiện mutual exclusion và đồng bộ hóa dữ liệu giữa các task. Các test case như Race Condition Test và

Mutex Test minh họa trực tiếp hiệu quả của các cơ chế này. Khi mutex được áp dụng, các task duy trì giá trị biến thống nhất; ngược lại, khi không sử dụng cơ chế bảo vệ phù hợp, dữ liệu có thể bị sai lệch do hiện tượng truy cập đồng thời.

Đồng thời, các tình huống deadlock và priority inversion cũng được tái hiện trong môi trường simulator. Ví dụ, khi hai task giữ mutex theo thứ tự ngược nhau, hệ thống mô phỏng cho thấy các task có thể bị chặn vô thời hạn. Việc áp dụng cơ chế priority inheritance hoặc cấu hình timeout cho mutex trong FreeRTOS giúp giảm thiểu các rủi ro này. Các log runtime thu thập trong quá trình chạy test case cung cấp thông tin về trạng thái task, thời điểm lock/unlock mutex, số lần context switch và cho phép nhóm đánh giá mức độ ổn định của hệ thống.

Từ góc độ kiểm thử, concurrency và synchronization không chỉ là các cơ chế kỹ thuật mà còn là đối tượng đánh giá quan trọng của đề tài. Việc phân tích hành vi của task trong các tình huống cạnh tranh tài nguyên giúp nhóm xác định tính đúng đắn của cơ chế đồng bộ hóa, khả năng duy trì dữ liệu nhất quán và mức độ ảnh hưởng của các lỗi đồng thời tới hiệu năng của hệ thống. Các kết quả này sẽ tiếp tục được sử dụng trong các chương về thiết kế kiểm thử, thu thập log và phân tích pass/fail.

Hình đề xuất

Hình 2.5. Đồng bộ hóa truy cập tài nguyên dùng chung bằng Mutex và Semaphore.

Hình 2.6. Trạng thái Blocked, Ready và Running của các task trong quá trình Synchronization.

2.5 Hard Deadline, Soft Deadline và Timing Analysis

Liên kết trực tiếp với các task đã triển khai trong Chương 1, khái niệm hard deadline và soft deadline quyết định cách đánh giá hiệu quả thời gian thực của hệ thống. Trong kịch bản **Hệ Thống Báo Cháy Thông Minh**, các task quan trọng như **Emergency_Task** phải hoàn thành trước hard deadline, bất kể trạng thái của các task khác. Trong khi đó, các task phụ trợ như **Logger_Task** hoặc **Communication_Task** trong **Trạm Quan Trắc Thời Tiết** có thể chậm hơn mà vẫn không ảnh hưởng tới chức năng chính, tạo thành các soft deadline.

Để đánh giá khả năng đáp ứng thời gian thực, đề tài sử dụng nhiều chỉ số timing metrics khác nhau, bao gồm **execution time**, **response time**, **latency**, **deadline miss**, **context switch count** và **Worst-Case Execution Time (WCET)**. Trong đó, WCET phản ánh thời gian thực thi lớn nhất của một task trong điều kiện tải cao hoặc khi xảy ra preemption, từ đó cung cấp cơ sở quan trọng để đánh giá khả năng đáp ứng deadline của

hệ thống. Các chỉ số này được thu thập từ FreeRTOS Simulator và sử dụng làm dữ liệu đầu vào cho quá trình timing analysis.

Ví dụ, khi **Emergency_Task** sẵn sàng chạy trong test case **Nhà Máy Có Emergency Stop**, scheduler cấp CPU gần như ngay lập tức và log ghi nhận response time dưới ngưỡng yêu cầu, đảm bảo hard deadline được đáp ứng. Ngược lại, trong test case **Trạm Quan Trắc Thời Tiết**, một task thuộc nhóm soft deadline có thể xuất hiện độ trễ lớn hơn mà không làm hệ thống mất chức năng cốt lõi. Việc so sánh execution time thực tế với WCET cũng cho phép nhóm đánh giá mức độ ổn định của task dưới các điều kiện vận hành khác nhau.

Timing analysis được thực hiện bằng cách phân tích log runtime, xác định thời điểm task bắt đầu và kết thúc, tính toán thời gian phản hồi, thời gian thực thi, độ trễ và tần suất context switch. Kết quả này được sử dụng để đánh giá khả năng đáp ứng deadline, hiệu quả của cơ chế preemption và tính ổn định của hệ thống khi nhiều task hoạt động đồng thời. Các bảng dữ liệu và biểu đồ Gantt, histogram hoặc response time distribution giúp minh họa trực quan kết quả kiểm thử.

Hình đề xuất

Hình 2.7. So sánh Hard Deadline và Soft Deadline trong các kịch bản kiểm thử.

Hình 2.8. Phân bố Response Time và Worst-Case Execution Time (WCET) của các task.

Hình 2.9. Gantt Chart thể hiện quá trình thực thi, preemption và deadline của các task.

2.6 Test Case, Test Suite và Log Analysis

Sau khi thiết lập các cơ sở lý thuyết liên quan đến scheduler, synchronization, deadline handling và timing analysis, việc tổ chức hoạt động kiểm thử trở thành bước cần thiết để đánh giá hành vi của hệ thống một cách có hệ thống. Trong các hệ thống nhúng thời gian thực, việc chương trình có thể thực thi thành công không đồng nghĩa với việc hệ thống đạt yêu cầu vận hành. Do đó, các cơ chế kiểm thử cần được xây dựng nhằm xác nhận tính đúng đắn của scheduler, khả năng đáp ứng deadline, hiệu quả đồng bộ hóa và mức độ ổn định của hệ thống trong nhiều điều kiện hoạt động khác nhau.

Trong phạm vi đề tài, hoạt động kiểm thử được tổ chức dựa trên **test case** và **test suite**. Test case được sử dụng để mô tả một tình huống kiểm thử cụ thể, bao gồm điều kiện thực thi, task tham gia, dữ liệu đầu vào, hành vi mong đợi và tiêu chí pass/fail. Nhiều test

case có cùng mục tiêu đánh giá được nhóm thành một test suite nhằm phục vụ kiểm thử theo chủ đề.

Trong phạm vi đề tài, tiêu chí pass/fail được xác định dựa trên khả năng đáp ứng hành vi mong đợi của scheduler, mức độ tuân thủ deadline, tính đúng đắn của cơ chế synchronization và các kết quả timing metrics thu được từ log runtime. Một test case được xem là đạt khi hệ thống thực hiện đúng trình tự lập lịch, đáp ứng deadline theo yêu cầu, duy trì tính nhất quán của dữ liệu và không xuất hiện các lỗi nghiêm trọng như deadlock hoặc race condition ngoài phạm vi kiểm thử. Ngược lại, test case được đánh giá là không đạt khi các chỉ số thu thập được vượt quá ngưỡng cho phép hoặc hành vi thực tế không phù hợp với kết quả mong đợi đã xác định trước.

Liên hệ với proposal của đề tài, các test suite được xây dựng xoay quanh các nhóm kiểm thử chính như **Scheduler Testing**, **Priority Scheduling**, **Priority Preemption**, **Synchronization**, **Concurrency**, **Hard Deadline**, **Soft Deadline**, **Fault Injection** và **Timing Analysis**. Cách tổ chức này cho phép nhóm đánh giá riêng biệt từng cơ chế của hệ thống đồng thời vẫn duy trì tính liên kết giữa các thành phần kiểm thử.

Trong môi trường **FreeRTOS Simulator**, các test case được triển khai thông qua sự phối hợp giữa các task chức năng, task giám sát deadline và task logging. Chẳng hạn, trong test case **Camera An Ninh Phát Hiện Xâm Nhập**, hệ thống kiểm tra khả năng priority preemption khi **Emergency_Task** xuất hiện trong lúc các task ưu tiên thấp đang hoạt động. Trong test case **Hệ Thống Báo Cháy Thông Minh**, việc đáp ứng hard deadline được đánh giá thông qua thời gian phản hồi và số lần deadline miss. Đối với các kịch bản **Trạm Quan Trắc Thời Tiết** hoặc **Hệ Thống Giám Sát Chất Lượng Không Khí**, nhóm phân tích mức độ đáp ứng soft deadline và tác động của tải hệ thống tới latency và response time.

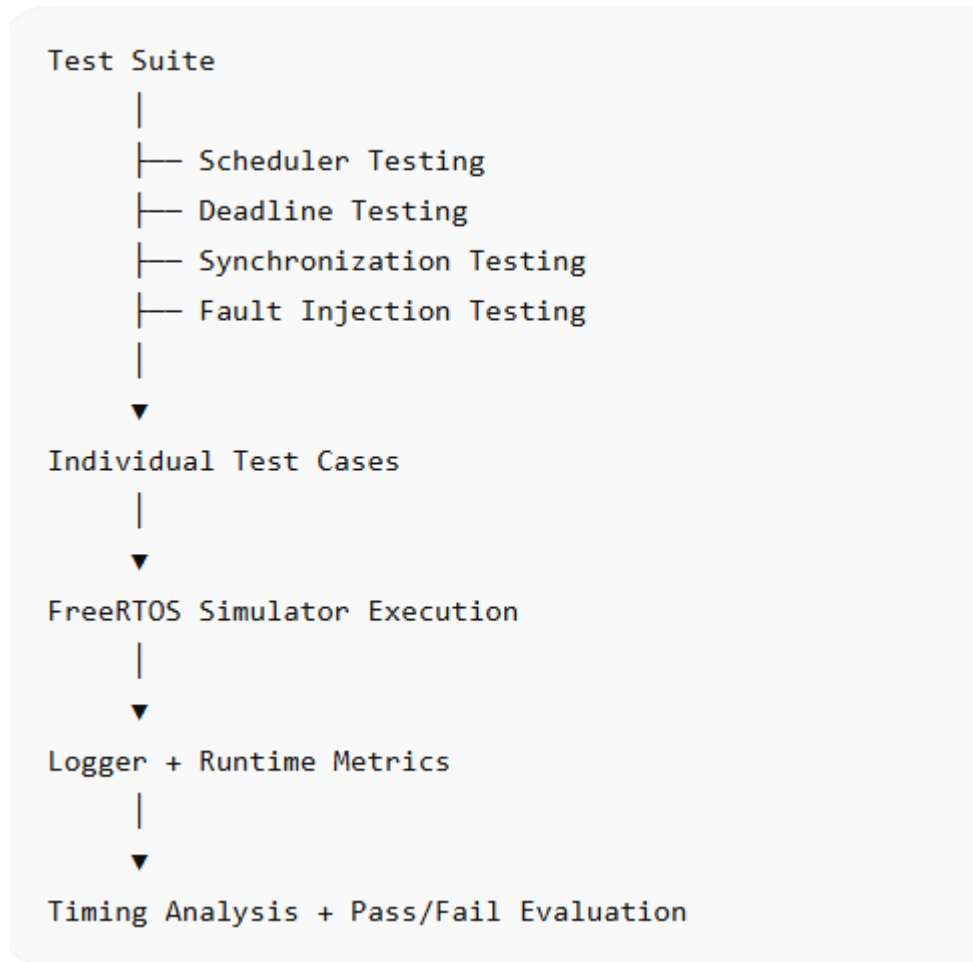
Để hỗ trợ quá trình đánh giá, hệ thống sử dụng cơ chế **log analysis** nhằm ghi nhận dữ liệu runtime trong quá trình thực thi. Logger_Task chịu trách nhiệm lưu lại thông tin liên quan đến trạng thái task, priority level, thời điểm context switch, thời gian bắt đầu thực thi, thời gian hoàn thành và các sự kiện synchronization. Các dữ liệu này tạo thành cơ sở cho việc tính toán các metrics như **execution time**, **response time**, **latency**, **deadline miss count** và **CPU utilization**.

Log analysis đóng vai trò đặc biệt quan trọng trong đề tài bởi đây là cơ chế kết nối trực tiếp giữa quá trình thực thi và hoạt động đánh giá pass/fail. Thay vì đánh giá hệ thống dựa trên quan sát thủ công, nhóm sử dụng dữ liệu runtime để xác minh tính đúng hạn của task, hiệu quả của scheduler, mức độ thành công của cơ chế synchronization và tác động của fault injection đối với hiệu năng hệ thống. Cách tiếp cận này giúp quá trình kiểm thử có tính lặp lại, định lượng được và phù hợp với mục tiêu kiểm thử hệ thống nhúng đa tiến trình thời gian thực.

Những cơ sở lý thuyết về test case, test suite và log analysis sẽ được sử dụng trực tiếp trong các chương tiếp theo để xây dựng bộ kiểm thử, thu thập dữ liệu thực nghiệm và phân tích kết quả vận hành của hệ thống FreeRTOS Simulator.

Gợi ý hình nên chèn

Hình 2.6. Quan hệ giữa Test Suite, Test Case và Log Analysis



2.7 Tổng kết Chương 2

Chương 2 đã trình bày toàn bộ cơ sở lý thuyết phục vụ trực tiếp cho việc xây dựng và kiểm thử hệ thống nhúng đa tiến trình thời gian thực sử dụng **FreeRTOS Simulator**. Bắt đầu từ kiến thức về **Embedded Systems**, chương làm rõ đặc điểm đa nhiệm, yêu cầu tài nguyên giới hạn và tầm quan trọng của thời gian phản hồi trong các thiết bị nhúng hiện đại. Tiếp theo, cơ chế **RTOS** được giới thiệu như một lớp trung gian quan trọng, chịu trách nhiệm quản lý scheduler, điều phối CPU, quản lý trạng thái task và đảm bảo tính đúng hạn, từ đó dẫn vào môi trường FreeRTOS mà nhóm sử dụng để mô phỏng hoạt động hệ thống.

Các cơ chế **scheduler**, **priority scheduling** và **priority preemption** được liên kết trực tiếp với các task đã thiết kế như **Sensor_Task**, **Control_Task** và **Emergency_Task**, đồng thời phản ánh trong các test case trọng tâm như **Camera An Ninh Phát Hiện Xâm Nhập** và **Nhà Máy Có Emergency Stop**. Cơ chế **concurrency** và **synchronization** cũng được triển khai thông qua mutex, semaphore, queue, nhằm đảm bảo việc truy cập tài nguyên dùng chung diễn ra an toàn, hạn chế hiện tượng race condition, deadlock và priority inversion.

Chương cũng làm rõ cách đánh giá thời gian thực thông qua các chỉ số **execution time**, **response time**, **latency**, **WCET**, **CPU usage**, **deadline miss** và **context switch**, cung cấp nền tảng cho việc thiết kế test case, thu thập log và phân tích pass/fail trong các chương tiếp theo. Các khái niệm về **Hard Deadline** và **Soft Deadline** được liên kết trực tiếp với các kịch bản thực tế như **Hệ Thống Báo Cháy Thông Minh** và **Trạm Quan Trắc Thời Tiết**, đồng thời phục vụ việc đánh giá hiệu năng và tuân thủ thời gian thực của hệ thống.

Tổng kết lại, các kiến thức trình bày trong chương này không chỉ cung cấp nền tảng lý thuyết mà còn là cầu nối trực tiếp giữa Chương 1 và các chương về **thiết kế hệ thống**, **cài đặt FreeRTOS**, **thiết kế kiểm thử** và **phân tích kết quả thực nghiệm**. Cấu trúc này đảm bảo rằng mọi cơ chế, test case và metrics trong đề án đều có cơ sở lý thuyết rõ ràng, giúp việc triển khai và phân tích kết quả trở nên nhất quán và minh bạch.

Gợi ý hình minh họa cuối chương:

- Diagram tổng hợp Chương 2: Embedded System → RTOS → FreeRTOS Simulator → Task Layer → Scheduler / Priority / Synchronization → Timing Analysis → Test Case → Pass/Fail Evaluation

Bảng 2.x. Liên hệ giữa cơ sở lý thuyết và hệ thống kiểm thử

Nội dung lý thuyết	Thành phần đồ án	Test Case liên quan
Scheduler	FreeRTOS Scheduler	Camera Security
Priority Scheduling	Emergency_Task	Emergency Stop
Synchronization	Mutex / Queue	Race Condition
Hard Deadline	Deadline Monitor	Fire Alarm

Soft Deadline	Logger_Task	Weather Station
Timing Analysis	Logger + Metrics	Timing Test

CHƯƠNG 3 – THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình thiết kế hệ thống kiểm thử được xây dựng trên nền tảng FreeRTOS Simulator nhằm phục vụ việc đánh giá các cơ chế của hệ thống nhúng đa tiến trình thời gian thực. Trên cơ sở lý thuyết đã trình bày trong Chương 2, nhóm tiến hành thiết kế kiến trúc tổng thể, xác định các task chức năng, cơ chế lập lịch, đồng bộ hóa tài nguyên, giám sát deadline và thu thập dữ liệu runtime.

Mục tiêu của chương là xây dựng một môi trường mô phỏng có khả năng tái hiện các đặc trưng thường gặp trong hệ thống nhúng thực tế, đồng thời hỗ trợ triển khai các test case liên quan đến scheduler, priority scheduling, priority preemption, synchronization, concurrency, fault injection và timing analysis. Các thành phần được thiết kế theo hướng phục vụ trực tiếp cho hoạt động kiểm thử và đánh giá pass/fail trong các chương tiếp theo.

3.1 Yêu cầu thiết kế hệ thống

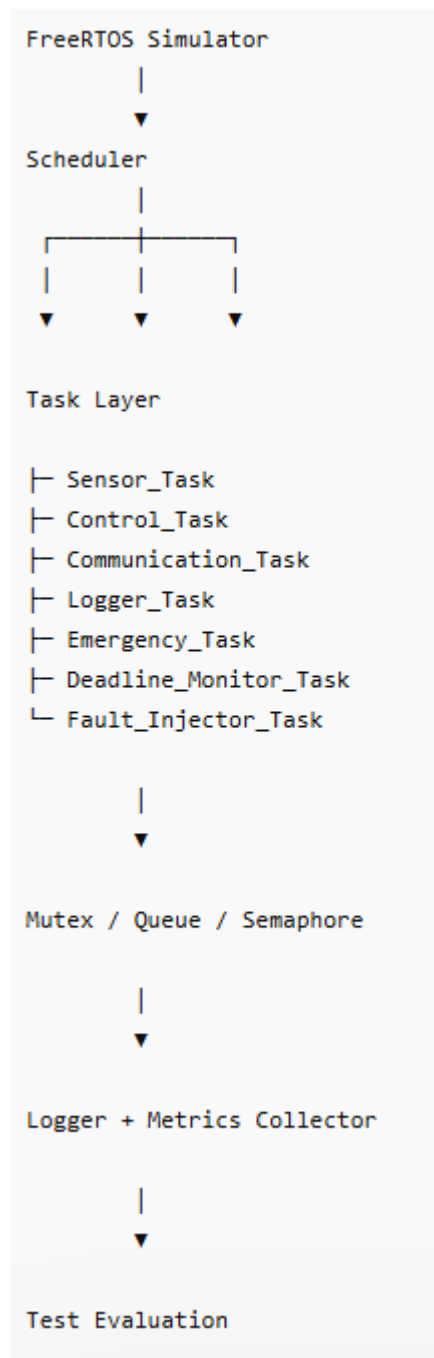
Yêu cầu	Mục đích
Multi-task Environment	Mô phỏng hệ thống nhúng thực tế
Priority Scheduling	Kiểm thử scheduler
Priority Preemption	Kiểm thử tác vụ khẩn cấp
Synchronization	Kiểm thử mutex và semaphore
Deadline Monitoring	Đánh giá hard/soft deadline
Fault Injection	Mô phỏng lỗi hệ thống
Runtime Logging	Thu thập dữ liệu phân tích
Metrics Collection	Đánh giá hiệu năng

Bảng 3.1. Yêu cầu thiết kế hệ thống

Xuất phát từ mục tiêu đã trình bày trong Chương 1 và các cơ sở lý thuyết ở Chương 2, hệ thống được thiết kế theo hướng ưu tiên khả năng kiểm thử hơn là phát triển một ứng dụng nhúng hoàn chỉnh. Do đó, kiến trúc hệ thống cần hỗ trợ đồng thời nhiều task, cho phép quan sát cơ chế scheduler, ghi nhận dữ liệu runtime và triển khai các tình huống kiểm thử liên quan đến deadline, synchronization và fault injection.

Bên cạnh việc mô phỏng môi trường thực thi của hệ thống nhúng thời gian thực, hệ thống còn phải cung cấp cơ chế logging và metrics collection nhằm phục vụ quá trình đánh giá pass/fail. Điều này cho phép các test case được thực hiện một cách có hệ thống và tạo cơ sở dữ liệu cho các phân tích ở chương sau.

3.2 Kiến trúc tổng thể FreeRTOS Simulator



Hình 3.1. Kiến trúc tổng thể hệ thống FreeRTOS Simulator

Kiến trúc hệ thống được xây dựng theo mô hình nhiều lớp nhằm tách biệt giữa môi trường thực thi, các task chức năng và cơ chế thu thập dữ liệu. FreeRTOS Simulator đóng vai trò môi trường thực thi trung tâm, nơi scheduler quản lý toàn bộ vòng đời của các task. Các task được thiết kế để mô phỏng các thành phần thường gặp trong hệ thống nhúng như cảm biến, điều khiển, truyền thông, ghi log và xử lý khẩn cấp.

Dưới lớp task là các cơ chế synchronization như mutex, semaphore và queue nhằm hỗ trợ trao đổi dữ liệu và điều phối truy cập tài nguyên dùng chung. Các sự kiện runtime

phát sinh trong quá trình thực thi được ghi nhận bởi Logger và Metrics Collector, từ đó tạo dữ liệu đầu vào cho các hoạt động timing analysis, deadline evaluation và pass/fail assessment.

3.3 Thiết kế các Task

Trong hệ thống FreeRTOS Simulator, task là đơn vị thực thi cơ bản được scheduler quản lý và phân bổ CPU. Dựa trên các yêu cầu thiết kế đã xác định ở mục 3.1 và kiến trúc tổng thể ở mục 3.2, nhóm xây dựng tập hợp các task nhằm mô phỏng các thành phần thường gặp trong hệ thống nhúng thời gian thực. Mỗi task được cấu hình với mức priority, chu kỳ thực thi (period), thời gian thực thi cực đại (Worst-Case Execution Time – WCET) và deadline riêng biệt để phục vụ các hoạt động kiểm thử scheduler, priority scheduling, priority preemption, synchronization, concurrency và timing analysis.

Việc phân tách hệ thống thành nhiều task độc lập cho phép mô phỏng môi trường đa nhiệm tương tự các hệ thống nhúng thực tế. Đồng thời, cách thiết kế này tạo điều kiện triển khai các test case theo nhiều kịch bản khác nhau như Camera An Ninh Phát Hiện Xâm Nhập, Hệ Thống Báo Cháy Thông Minh, Drone Tránh Chướng Ngại Vật, Trạm Quan Trắc Thời Tiết và Nhà Máy Có Emergency Stop.

Bảng 3.2. Cấu hình các Task trong hệ thống

Task	Priority	Period	WCET	Deadline
Emergency_Task	5	Event Driven	10 ms	20 ms
Control_Task	4	50 ms	15 ms	50 ms
Deadline_Monitor_Task	4	100 ms	10 ms	100 ms
Sensor_Task	3	100 ms	10 ms	100 ms
Communication_Task	2	150 ms	15 ms	150 ms
Fault_Injector_Task	2	500 ms	20 ms	500 ms
Logger_Task	1	200 ms	20 ms	200 ms

Các giá trị period, WCET và deadline được lựa chọn nhằm tạo ra môi trường kiểm thử đa dạng, trong đó tồn tại đồng thời các task có mức ưu tiên khác nhau, các nhiệm vụ có hard deadline và soft deadline, cũng như các tình huống preemption và synchronization thường gặp trong hệ thống thời gian thực.

Sensor_Task: Sensor_Task mô phỏng hoạt động của các cảm biến trong hệ thống nhúng thực tế, có nhiệm vụ tạo dữ liệu đầu vào định kỳ và gửi tới các task xử lý thông qua queue. Task này được sử dụng trong các kịch bản như Drone Tránh Chướng Ngại Vật, Trạm Quan Trắc Thời Tiết và Hệ Thống Giám Sát Chất Lượng Không Khí, đồng thời hỗ trợ kiểm thử scheduler, queue communication và synchronization giữa các task.

Control_Task: Control_Task chịu trách nhiệm xử lý dữ liệu nhận từ Sensor_Task và đưa ra quyết định điều khiển tương ứng. Đây là task trung tâm của hệ thống, được sử dụng trong các kịch bản yêu cầu phản hồi thời gian thực như Camera An Ninh Phát Hiện Xâm Nhập hoặc Drone Tránh Chướng Ngại Vật, đồng thời là đối tượng đánh giá response time và timing analysis.

Communication_Task: Communication_Task thực hiện chức năng truyền dữ liệu và trạng thái hệ thống tới các thành phần bên ngoài. Task này được sử dụng để đánh giá ảnh hưởng của scheduler tới các tác vụ truyền thông và kiểm tra khả năng duy trì hoạt động ổn định khi hệ thống chịu tải.

Logger_Task: Logger_Task đảm nhiệm việc thu thập dữ liệu runtime phát sinh trong quá trình thực thi hệ thống. Các thông tin như execution time, response time, context switch, deadline miss và các sự kiện synchronization được ghi nhận thông qua task này, tạo cơ sở cho hoạt động phân tích và đánh giá pass/fail.

Emergency_Task: Emergency_Task mô phỏng các tình huống khẩn cấp như phát hiện cháy, phát hiện xâm nhập hoặc dừng khẩn cấp trong nhà máy. Đây là task có mức priority cao nhất và là đối tượng chính của các test case liên quan đến Priority Scheduling, Priority Preemption và Hard Deadline.

Deadline_Monitor_Task: Deadline_Monitor_Task có nhiệm vụ giám sát thời gian thực thi của các task khác và phát hiện các trường hợp vi phạm deadline. Task này hỗ trợ trực tiếp cho các hoạt động Deadline Testing, Timing Analysis và Pass/Fail Evaluation trong hệ thống.

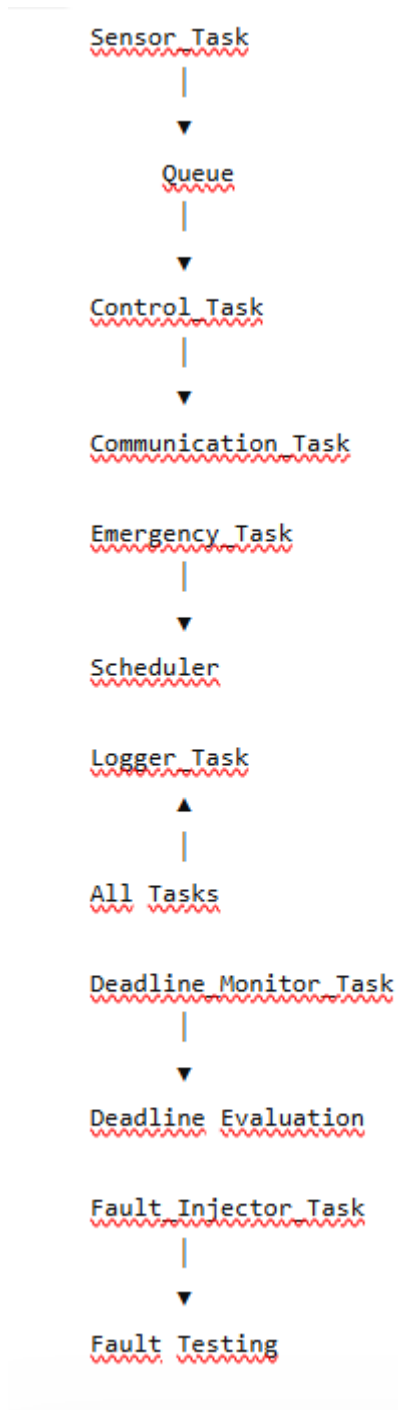
Fault_Injector_Task: Fault_Injector_Task được xây dựng nhằm tạo ra các tình huống lỗi có kiểm soát như CPU overload, artificial delay, queue blocking hoặc race condition. Task này cho phép đánh giá khả năng chịu lỗi của scheduler và tác động của các fault tới response time, deadline và tính ổn định của hệ thống.

Nhìn chung, tập hợp các task được thiết kế nhằm tái hiện những thành phần quan trọng của hệ thống nhúng thời gian thực, đồng thời tạo môi trường phù hợp để triển khai toàn bộ các nhóm kiểm thử đã đề xuất trong proposal. Cấu trúc này không chỉ hỗ trợ đánh giá scheduler, synchronization và deadline handling mà còn tạo nền tảng cho việc thu thập dữ liệu runtime và phân tích pass/fail trong các chương tiếp theo.

Hình 3.2. Mối quan hệ giữa các task trong FreeRTOS Simulator

Hình đề xuất

Hình 3.2. Mối quan hệ giữa các task trong FreeRTOS Simulator



3.4 Thiết kế Scheduler và Priority

Sau khi xác định các task chức năng của hệ thống, bước tiếp theo là xây dựng cơ chế lập lịch nhằm đảm bảo các task được thực thi theo đúng mức độ ưu tiên và yêu cầu thời gian thực. Trong hệ thống FreeRTOS Simulator, scheduler đóng vai trò trung tâm trong việc quản lý trạng thái task, phân bổ CPU và thực hiện preemption khi xuất hiện các tác vụ có mức ưu tiên cao hơn.

Việc thiết kế scheduler của đề tài bám sát các mục tiêu đã trình bày trong proposal, đặc biệt là các nhóm kiểm thử liên quan đến Scheduler Testing, Priority Scheduling, Priority Preemption, Hard Deadline và Timing Analysis. Do đó, cơ chế lập lịch không chỉ phục vụ hoạt động vận hành của hệ thống mà còn đóng vai trò đối tượng chính trong nhiều test case thực nghiệm.

Trong FreeRTOS, scheduler sử dụng mô hình priority-based preemptive scheduling, trong đó task có mức priority cao nhất trong trạng thái Ready sẽ được cấp CPU để thực thi. Khi xuất hiện một task có priority cao hơn task đang chạy, scheduler sẽ thực hiện context switch và chuyển quyền xử lý sang task ưu tiên cao hơn. Cơ chế này đặc biệt phù hợp với các hệ thống nhúng thời gian thực, nơi các nhiệm vụ khẩn cấp cần được xử lý ngay lập tức.

Bảng 3.3. Phân bổ mức ưu tiên cho các task

Task	Priority
Emergency_Task	5
Control_Task	4
Deadline_Monitor_Task	4
Sensor_Task	3
Communication_Task	2
Fault_Injector_Task	2
Logger_Task	1

Việc phân bổ priority được thực hiện dựa trên mức độ quan trọng của từng task đối với hoạt động của hệ thống. Các task liên quan trực tiếp đến an toàn hoặc điều khiển được ưu tiên cao hơn các task phục vụ ghi log hoặc truyền thông nhằm đảm bảo khả năng đáp ứng thời gian thực.

Emergency_Task được gán mức priority cao nhất nhằm mô phỏng các tình huống khẩn cấp như phát hiện cháy, phát hiện xâm nhập hoặc dừng khẩn cấp trong nhà máy. Trong các kịch bản này, hệ thống phải phản ứng ngay lập tức khi xuất hiện sự kiện kích hoạt. Việc đặt Emergency_Task ở mức priority cao nhất cho phép kiểm thử trực tiếp cơ chế priority preemption của FreeRTOS và đánh giá khả năng phản ứng của scheduler trong các tình huống thời gian thực nghiêm ngặt.

Control_Task và Deadline_Monitor_Task được gán mức priority cao thứ hai. Control_Task chịu trách nhiệm xử lý dữ liệu và đưa ra quyết định điều khiển, trong khi Deadline_Monitor_Task giám sát khả năng đáp ứng deadline của toàn hệ thống. Hai task này có ảnh hưởng trực tiếp tới hoạt động vận hành nên cần được ưu tiên cao hơn các task truyền thông hoặc ghi log.

Sensor_Task được cấu hình ở mức priority trung bình. Nhiệm vụ chính của task này là tạo dữ liệu đầu vào cho hệ thống. Mặc dù đóng vai trò quan trọng, Sensor_Task không cần mức ưu tiên cao hơn các tác vụ điều khiển hoặc xử lý khẩn cấp vì dữ liệu cảm biến vẫn phải được xử lý bởi các task cấp trên trước khi tạo ra hành động điều khiển.

Communication_Task và Fault_Injector_Task được đặt ở mức priority thấp hơn. Communication_Task chủ yếu thực hiện trao đổi dữ liệu với các thành phần bên ngoài và không trực tiếp ảnh hưởng tới khả năng điều khiển tức thời của hệ thống.

Fault_Injector_Task chỉ hoạt động trong các kịch bản kiểm thử nên không được phép ảnh hưởng tới hoạt động bình thường của các task quan trọng.

Logger_Task được gán mức priority thấp nhất nhằm giảm thiểu tác động tới hoạt động của scheduler. Mặc dù đóng vai trò quan trọng trong việc thu thập dữ liệu runtime, việc ghi log không được phép ảnh hưởng tới các nhiệm vụ điều khiển hoặc các task có deadline nghiêm ngặt.

Thông qua cách phân bổ priority này, hệ thống có thể tái hiện các tình huống thường gặp trong hệ thống nhúng thực tế, nơi các tác vụ khẩn cấp phải được ưu tiên xử lý trước các nhiệm vụ nền. Điều này tạo điều kiện thuận lợi để triển khai các test case liên quan đến priority scheduling, priority preemption và deadline handling.

Thiết kế cơ chế Priority Preemption

Một trong những mục tiêu quan trọng của đề tài là đánh giá khả năng preemption của scheduler khi xuất hiện task ưu tiên cao. Trong hệ thống được thiết kế, cơ chế này được kiểm tra chủ yếu thông qua Emergency_Task.

Khi hệ thống đang thực thi Sensor_Task, Communication_Task hoặc Logger_Task và xuất hiện tín hiệu khẩn cấp, Emergency_Task sẽ chuyển sang trạng thái Ready. Scheduler phải ngay lập tức thực hiện context switch và cấp CPU cho Emergency_Task. Sau khi xử lý xong sự kiện khẩn cấp, scheduler mới tiếp tục thực thi các task bị gián đoạn.

Cơ chế này được sử dụng trong các test case như Nhà Máy Có Emergency Stop, Camera An Ninh Phát Hiện Xâm Nhập và Hệ Thống Báo Cháy Thông Minh nhằm đánh giá khả năng phản ứng của scheduler và xác minh tính đúng đắn của priority preemption.

Thiết kế trạng thái Task

Trong quá trình vận hành, các task có thể chuyển đổi giữa nhiều trạng thái khác nhau dưới sự quản lý của scheduler. Các trạng thái chính được sử dụng trong hệ thống bao gồm Ready, Running, Blocked và Suspended.

Ready biểu thị task đã sẵn sàng thực thi và đang chờ được cấp CPU. Running biểu thị task đang sử dụng CPU. Blocked xuất hiện khi task chờ dữ liệu từ queue, semaphore hoặc delay timer. Suspended được sử dụng khi task bị tạm dừng và không tham gia quá trình lập lịch.

Việc theo dõi các trạng thái này cho phép nhóm phân tích hành vi của scheduler thông qua log runtime và hỗ trợ các hoạt động timing analysis ở Chương 6.

Hình đề xuất

Hình 3.3. Priority Assignment của các task trong hệ thống

```
Priority 5 → Emergency_Task

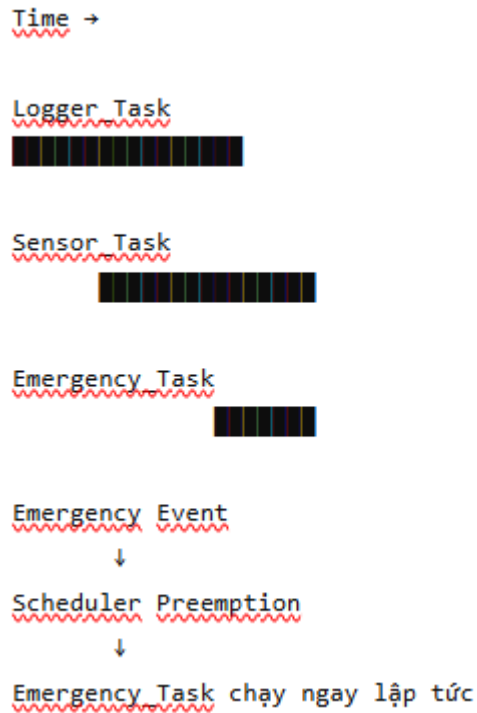
Priority 4 → Control_Task
          → Deadline_Monitor_Task

Priority 3 → Sensor_Task

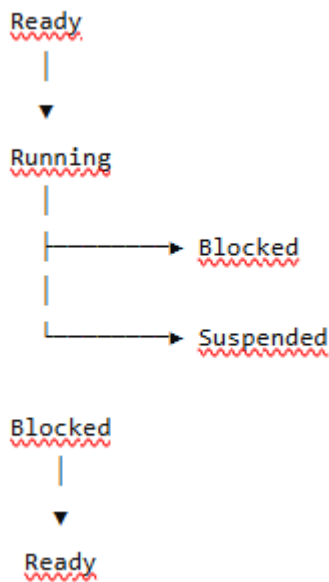
Priority 2 → Communication_Task
          → Fault_Injector_Task

Priority 1 → Logger_Task
```

Hình 3.4. Cơ chế Priority Preemption



Hình 3.5. Sơ đồ chuyển trạng thái Task trong FreeRTOS



3.5 Thiết kế Synchronization và Shared Resource

Trong quá trình vận hành, nhiều task trong hệ thống cần trao đổi dữ liệu hoặc truy cập các tài nguyên dùng chung. Để đảm bảo tính nhất quán của dữ liệu và tránh các lỗi phát sinh do thực thi đồng thời, hệ thống được thiết kế sử dụng các cơ chế synchronization của FreeRTOS như mutex, semaphore và queue. Các cơ chế này được triển khai trực tiếp trong

kiến trúc hệ thống nhằm phục vụ các hoạt động kiểm thử liên quan đến synchronization, concurrency và timing analysis.

Trong hệ thống được thiết kế, mutex được sử dụng để bảo vệ các tài nguyên dùng chung như Shared Log Buffer và System Status Variable. Khi nhiều task cùng thực hiện thao tác ghi log hoặc cập nhật trạng thái hệ thống, mutex đảm bảo chỉ một task được phép truy cập tài nguyên tại một thời điểm. Điều này giúp duy trì tính nhất quán của dữ liệu và tạo điều kiện triển khai các test case liên quan đến race condition, deadlock và priority inversion.

Queue được sử dụng làm cơ chế trao đổi dữ liệu chính giữa các task. Sensor_Task gửi dữ liệu cảm biến tới Control_Task thông qua hàng đợi, trong khi Control_Task có thể tiếp tục chuyển kết quả xử lý tới Communication_Task hoặc Logger_Task. Việc sử dụng queue giúp tách biệt giữa quá trình tạo dữ liệu và quá trình xử lý dữ liệu, đồng thời giảm sự phụ thuộc trực tiếp giữa các task.

Semaphore được sử dụng để đồng bộ hóa các sự kiện phát sinh trong hệ thống. Trong các kịch bản khẩn cấp, Sensor_Task hoặc Deadline_Monitor_Task có thể phát tín hiệu thông qua semaphore để kích hoạt Emergency_Task hoặc Logger_Task thực hiện các hành động tương ứng. Cơ chế này giúp giảm thời gian phản hồi của hệ thống và hỗ trợ triển khai các test case liên quan đến synchronization và deadline handling.

Bảng 3.4. Tài nguyên dùng chung và cơ chế bảo vệ

Tài nguyên	Task liên quan	Cơ chế bảo vệ
Sensor Data Queue	Sensor_Task, Control_Task	Queue
Communication Buffer	Control_Task, Communication_Task	Queue
System Status Variable	Control_Task, Emergency_Task	Mutex
Shared Log Buffer	Logger_Task, All Tasks	Mutex
Event Notification	Sensor_Task, Emergency_Task	Semaphore
Deadline Alert Signal	Deadline_Monitor_Task, Logger_Task	Semaphore

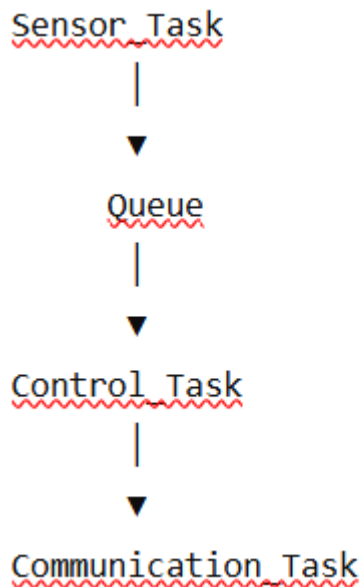
Thông qua việc xác định rõ tài nguyên dùng chung và cơ chế bảo vệ tương ứng, hệ thống có thể duy trì tính nhất quán của dữ liệu trong môi trường đa nhiệm, đồng thời tạo điều kiện thuận lợi cho việc triển khai các kịch bản kiểm thử liên quan đến synchronization.

Để đánh giá khả năng xử lý đồng thời của hệ thống, nhiều task được thiết kế hoạt động song song trong cùng khoảng thời gian. Các tình huống này cho phép kiểm tra hiệu quả của scheduler cũng như cơ chế synchronization khi xuất hiện tranh chấp tài nguyên. Trong các test case Synchronization Testing và Concurrency Testing, dữ liệu runtime được thu thập để đánh giá thời gian chờ mutex, số lần truy cập queue, thời gian phản hồi của semaphore và tác động của các cơ chế đồng bộ hóa tới hiệu năng hệ thống.

Những dữ liệu thu thập được từ các kịch bản này sẽ được sử dụng trong Chương 5 và Chương 6 để đánh giá tính đúng đắn của cơ chế synchronization, khả năng duy trì dữ liệu nhất quán và mức độ ổn định của hệ thống trong điều kiện hoạt động đồng thời.

Hình đề xuất

Hình 3.6. Mô hình trao đổi dữ liệu giữa các task thông qua Queue



3.6 Thiết kế Deadline Monitoring

Một trong những yêu cầu quan trọng nhất của hệ thống nhúng thời gian thực là khả năng hoàn thành nhiệm vụ trong khoảng thời gian cho phép. Trong nhiều ứng dụng thực tế, việc đưa ra kết quả chính xác nhưng không đúng thời điểm vẫn được xem là lỗi hệ thống. Vì vậy, bên cạnh cơ chế scheduler và synchronization, hệ thống cần có khả năng giám sát deadline nhằm đánh giá mức độ tuân thủ các ràng buộc thời gian của từng task.

Xuất phát từ các mục tiêu kiểm thử đã trình bày trong proposal, hệ thống được thiết kế để hỗ trợ đánh giá cả Hard Deadline và Soft Deadline. Cơ chế này đóng vai trò trung tâm trong các nhóm kiểm thử Deadline Testing, Timing Analysis và Pass/Fail

Evaluation, đồng thời cung cấp dữ liệu quan trọng cho việc đánh giá hiệu năng của scheduler.

Để thực hiện chức năng này, hệ thống sử dụng `Deadline_Monitor_Task` nhằm theo dõi thời gian thực thi, thời gian phản hồi và trạng thái hoàn thành của các task trong quá trình vận hành. Các thông tin thu thập được sẽ được so sánh với deadline đã cấu hình để xác định các trường hợp vi phạm thời gian thực.

Thiết kế Hard Deadline

Hard Deadline được sử dụng cho các nhiệm vụ mà việc hoàn thành muộn được xem là lỗi nghiêm trọng của hệ thống. Trong các hệ thống nhúng thực tế, hard deadline thường xuất hiện trong các ứng dụng liên quan đến an toàn, bảo vệ con người hoặc điều khiển thiết bị quan trọng.

Trong phạm vi đề tài, các kịch bản như Hệ Thống Báo Cháy Thông Minh, Camera An Ninh Phát Hiện Xâm Nhập và Nhà Máy Có Emergency Stop được xem là các tình huống có hard deadline. Khi xuất hiện sự kiện khẩn cấp, `Emergency_Task` phải được scheduler cấp CPU và hoàn thành xử lý trước thời hạn quy định. Nếu task hoàn thành sau deadline, test case sẽ được đánh giá là Fail bất kể kết quả xử lý có chính xác hay không.

Thiết kế Soft Deadline

Khác với hard deadline, soft deadline cho phép một mức độ trễ nhất định mà không gây ảnh hưởng nghiêm trọng tới chức năng cốt lõi của hệ thống. Các nhiệm vụ thuộc nhóm này vẫn cần được thực hiện đúng thời gian càng nhiều càng tốt, tuy nhiên việc vi phạm deadline không dẫn tới lỗi hệ thống nghiêm trọng.

Trong hệ thống được thiết kế, `Logger_Task` và `Communication_Task` được xem là các task thuộc nhóm soft deadline. Các task này phục vụ mục đích ghi nhận dữ liệu hoặc truyền thông nên có thể chấp nhận độ trễ nhỏ trong điều kiện tải hệ thống tăng cao.

Việc phân tách hard deadline và soft deadline cho phép hệ thống đánh giá chính xác hơn mức độ ảnh hưởng của từng trường hợp deadline miss và phản ánh sát hơn các điều kiện vận hành thực tế.

Bảng 3.5. Phân loại Deadline của các Task

Task	Deadline	Loại Deadline
<code>Emergency_Task</code>	20 ms	Hard
<code>Control_Task</code>	50 ms	Hard
<code>Deadline_Monitor_Task</code>	100 ms	Hard
<code>Sensor_Task</code>	100 ms	Soft
<code>Communication_Task</code>	150 ms	Soft

Logger_Task	200 ms	Soft
Fault_Injector_Task	500 ms	Soft

Bảng trên thể hiện các ngưỡng thời gian được sử dụng trong hệ thống. Những giá trị này được lựa chọn nhằm tạo điều kiện kiểm thử nhiều tình huống deadline khác nhau, từ các nhiệm vụ yêu cầu phản ứng tức thời tới các nhiệm vụ nền có mức độ ưu tiên thấp hơn.

Thiết kế Deadline Miss Detection

Để hỗ trợ các hoạt động kiểm thử, hệ thống được trang bị cơ chế phát hiện deadline miss. Deadline_Monitor_Task định kỳ kiểm tra thời gian hoàn thành của các task và so sánh với deadline tương ứng.

Khi một task vượt quá thời hạn quy định, hệ thống sẽ ghi nhận sự kiện deadline miss vào log runtime. Các thông tin như tên task, thời điểm xảy ra vi phạm, thời gian thực thi thực tế và độ trễ vượt ngưỡng sẽ được lưu trữ để phục vụ quá trình phân tích sau này.

Cơ chế này cho phép nhóm đánh giá hiệu quả của scheduler trong nhiều điều kiện hoạt động khác nhau, đồng thời hỗ trợ các test case về CPU overload, priority preemption và fault injection.

Thiết kế Timing Metrics

Bên cạnh việc xác định deadline miss, hệ thống còn thu thập nhiều chỉ số thời gian nhằm phục vụ hoạt động timing analysis.

Bảng 3.6. Các chỉ số thời gian được giám sát

Chỉ số	Ý nghĩa
Execution Time	Thời gian thực thi của task
Response Time	Thời gian từ khi task sẵn sàng đến khi bắt đầu chạy
Latency	Độ trễ xử lý của hệ thống
WCET	Thời gian thực thi lớn nhất của task
Deadline Miss Count	Số lần vi phạm deadline
Context Switch Count	Số lần chuyển ngữ cảnh
CPU Usage	Mức sử dụng CPU

Các chỉ số này được thu thập trong suốt quá trình chạy test và trở thành cơ sở cho các hoạt động đánh giá pass/fail ở Chương 5 và phân tích thực nghiệm ở Chương 6.

Thiết kế Pass/Fail Evaluation

Trong phạm vi đề tài, kết quả kiểm thử liên quan đến deadline được đánh giá dựa trên mức độ tuân thủ các ngưỡng thời gian đã xác định trước. Một test case được xem là Pass khi task hoàn thành trong khoảng thời gian cho phép và không xuất hiện deadline miss ngoài ngưỡng chấp nhận.

Ngược lại, test case sẽ được đánh giá là Fail nếu task vi phạm hard deadline hoặc số lượng deadline miss vượt quá tiêu chí cho phép. Đối với các soft deadline, việc đánh giá được thực hiện dựa trên tỷ lệ đáp ứng deadline và mức độ ảnh hưởng tới hoạt động chung của hệ thống.

Cách tiếp cận này cho phép quá trình đánh giá được thực hiện dựa trên dữ liệu định lượng thay vì quan sát thủ công, đồng thời phản ánh đúng bản chất của các hệ thống nhúng thời gian thực.

Hình đề xuất

Hình 3.9. Kiến trúc giám sát Deadline trong hệ thống

Hình 3.10. Hard Deadline và Soft Deadline

Hình 3.11. Quy trình phát hiện Deadline Miss

3.7 Thiết kế Logging và Metrics Collection

Để phục vụ hoạt động kiểm thử và đánh giá hiệu năng của hệ thống, việc thu thập dữ liệu runtime được xem là một thành phần quan trọng trong kiến trúc FreeRTOS Simulator. Thay vì đánh giá hệ thống thông qua quan sát thủ công, đề tài sử dụng cơ chế logging nhằm ghi nhận các sự kiện phát sinh trong quá trình thực thi và hỗ trợ phân tích hành vi của scheduler, synchronization và deadline monitoring.

Trong hệ thống được thiết kế, Logger_Task chịu trách nhiệm thu thập và lưu trữ dữ liệu runtime từ các task khác. Mỗi khi xảy ra các sự kiện như task được scheduler cấp CPU, task hoàn thành xử lý, semaphore được giải phóng hoặc xuất hiện context switch, hệ thống sẽ tạo bản ghi log tương ứng. Các bản ghi này được lưu theo timestamp nhằm phục vụ việc phân tích trình tự thực thi và đánh giá kết quả kiểm thử.

Bên cạnh các chỉ số timing đã được trình bày ở mục 3.6, hệ thống còn thu thập một số metrics bổ sung nhằm đánh giá hiệu năng tổng thể và mức độ sử dụng tài nguyên trong quá trình vận hành.

Bảng 3.7. Các Metrics bổ sung được thu thập

Metric	Ý nghĩa
CPU Usage	Mức sử dụng CPU của hệ thống
Context Switch Count	Số lần chuyển ngữ cảnh giữa các task

Queue Length	Số lượng phân tử trong queue
Semaphore Wait Time	Thời gian chờ semaphore hoặc mutex

Các chỉ số này cho phép đánh giá mức độ tải của hệ thống, hiệu quả hoạt động của scheduler và tác động của các cơ chế synchronization tới hiệu năng chung.

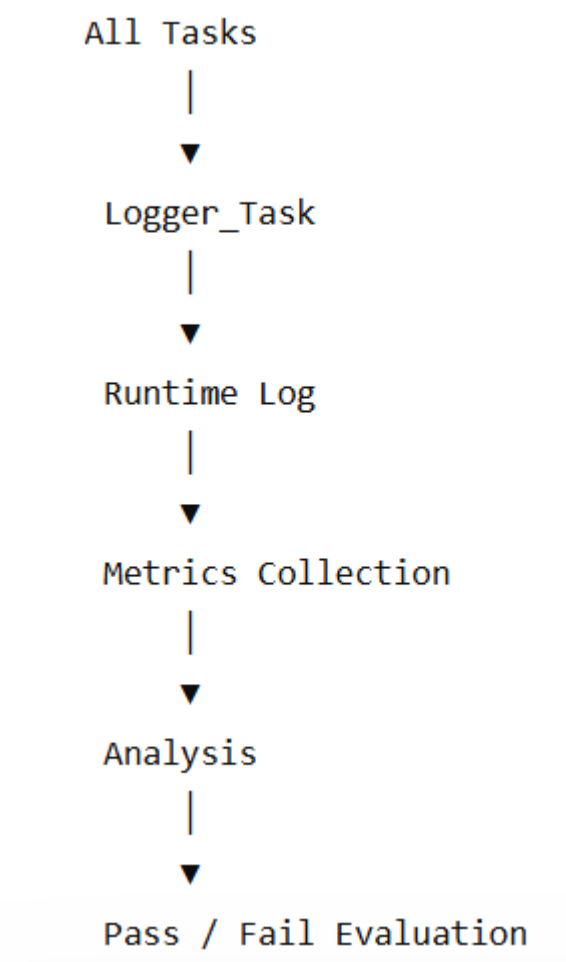
Bảng 3.8. Định dạng Log Runtime

Timestamp	Task	Event	Value
120 ms	Sensor_Task	Queue Send	Success
125 ms	Control_Task	Task Start	-
140 ms	Control_Task	Task Finish	15 ms
145 ms	Emergency_Task	Preemption	Triggered
160 ms	Deadline_Monitor_Task	Deadline Check	Pass

Thông qua cấu trúc log thống nhất, dữ liệu runtime có thể được xử lý tự động và sử dụng trong các hoạt động phân tích scheduler, synchronization, fault injection và pass/fail evaluation. Đồng thời, các dữ liệu này cũng là nguồn thông tin đầu vào cho các biểu đồ và bảng kết quả thực nghiệm được trình bày trong Chương 6.

Hình đề xuất

Hình 3.12. Luồng thu thập dữ liệu Runtime



3.8 Thiết kế Fault Injection

Bên cạnh việc đánh giá hệ thống trong điều kiện vận hành bình thường, đề tài còn xem xét khả năng hoạt động của hệ thống khi xuất hiện các điều kiện bất lợi hoặc lỗi bất thường. Để thực hiện điều này, hệ thống được trang bị cơ chế fault injection nhằm tạo ra các tình huống lỗi có kiểm soát và quan sát phản ứng của scheduler cũng như các task liên quan.

Fault injection cho phép đánh giá khả năng chịu lỗi của hệ thống, xác định giới hạn hoạt động của scheduler và phân tích ảnh hưởng của lỗi tới các chỉ số timing metrics. Đây là một trong những nội dung quan trọng được đề xuất trong proposal nhằm tăng tính thực tiễn của hoạt động kiểm thử.

Trong hệ thống, Fault_Injector_Task chịu trách nhiệm tạo ra các lỗi giả lập tại các thời điểm xác định trước hoặc theo kịch bản kiểm thử. Các lỗi được thiết kế nhằm tác động tới scheduler, synchronization hoặc deadline handling.

Bảng 3.9. Fault Injection Matrix

Fault Scenario	Mục tiêu kiểm thử
CPU Overload	Đánh giá Scheduler
Artificial Delay	Kiểm tra Deadline
Queue Blocking	Đánh giá Synchronization
Race Condition	Đánh giá Concurrency
Priority Inversion	Đánh giá Priority Handling
Burst Event	Kiểm tra khả năng phản ứng của Scheduler
Resource Contention	Đánh giá Mutex Handling

Ví dụ, trong kịch bản CPU Overload, Fault_Injector_Task tạo tải xử lý lớn nhằm quan sát khả năng duy trì deadline của các task quan trọng. Trong kịch bản Priority Inversion, các task có priority khác nhau cùng tranh chấp một tài nguyên được bảo vệ bằng mutex nhằm kiểm tra hiệu quả của cơ chế priority inheritance.

Thông qua các tình huống fault injection, hệ thống có thể được đánh giá trong nhiều điều kiện vận hành khác nhau, từ đó phản ánh sát hơn hành vi của các hệ thống nhúng thực tế.

Hình đề xuất

Hình 3.13. Mô hình Fault Injection

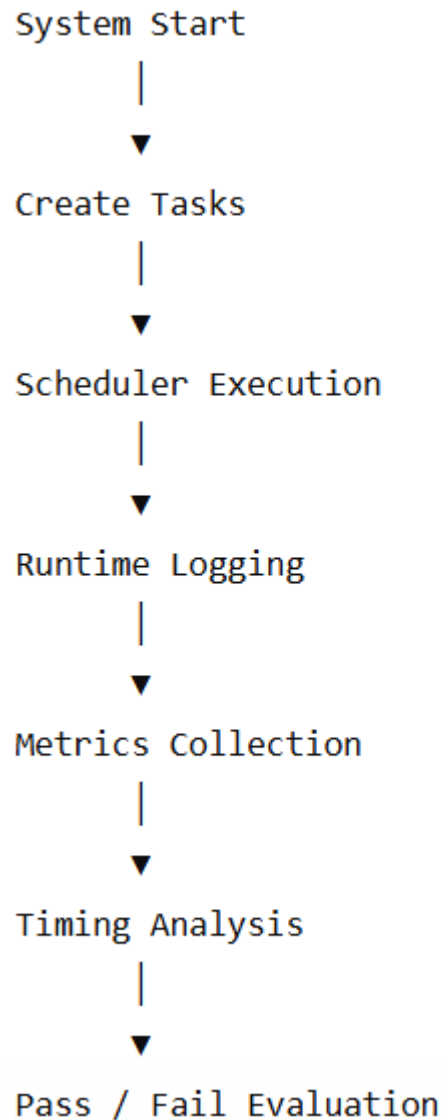
3.9 Workflow Kiểm thử

Sau khi hoàn thiện các thành phần chức năng, scheduler, synchronization, deadline monitoring, logging và fault injection, hệ thống được tổ chức thành một quy trình kiểm thử thống nhất nhằm hỗ trợ việc triển khai các test case trong chương tiếp theo.

Workflow kiểm thử mô tả toàn bộ quá trình từ khởi tạo hệ thống, thực thi task, thu thập dữ liệu runtime cho tới đánh giá pass/fail. Việc xây dựng workflow rõ ràng giúp đảm bảo mọi test case đều được thực hiện theo cùng một quy trình và tạo điều kiện thuận lợi cho việc tái hiện kết quả thực nghiệm.

Trong mỗi lần chạy kiểm thử, hệ thống khởi tạo các task theo cấu hình đã xác định, kích hoạt scheduler và bắt đầu thu thập dữ liệu runtime. Sau khi hoàn thành test case, các metrics được tổng hợp và so sánh với tiêu chí đánh giá tương ứng nhằm xác định kết quả pass hoặc fail.

Hình 3.14. Workflow kiểm thử tổng thể



Workflow này được sử dụng cho toàn bộ các nhóm kiểm thử như Scheduler Testing, Priority Scheduling, Synchronization Testing, Fault Injection Testing và Timing Analysis.

3.10 Tổng kết chương

Chương 3 đã trình bày quá trình thiết kế hệ thống kiểm thử được xây dựng trên nền tảng FreeRTOS Simulator nhằm phục vụ việc đánh giá các cơ chế của hệ thống nhúng đa tiến trình thời gian thực. Trên cơ sở các yêu cầu đã xác định ở Chương 1 và các nền tảng lý thuyết ở Chương 2, nhóm đã xây dựng kiến trúc tổng thể của hệ thống, xác định các task chức năng và thiết kế cơ chế lập lịch dựa trên mức độ ưu tiên.

Bên cạnh đó, chương cũng trình bày cách thức tổ chức các cơ chế synchronization, quản lý tài nguyên dùng chung, giám sát deadline và thu thập dữ liệu runtime. Các thành

phần này đóng vai trò quan trọng trong việc triển khai các test case liên quan đến scheduler, priority scheduling, concurrency, synchronization, hard deadline và soft deadline.

Ngoài các chức năng vận hành thông thường, hệ thống còn được bổ sung cơ chế fault injection nhằm tạo ra các điều kiện kiểm thử bất lợi và đánh giá khả năng chịu lỗi của scheduler. Các dữ liệu thu thập từ Logger_Task và Metrics Collector tạo nền tảng cho việc phân tích timing metrics và đánh giá pass/fail trong các chương tiếp theo.

Nhìn chung, kiến trúc được thiết kế không chỉ phục vụ mục đích mô phỏng hệ thống nhúng thời gian thực mà còn hỗ trợ trực tiếp cho hoạt động kiểm thử theo định hướng của proposal. Trên cơ sở thiết kế này, Chương 4 sẽ trình bày quá trình cài đặt hệ thống bằng FreeRTOS Simulator, bao gồm việc tạo task, cấu hình scheduler, triển khai mutex, semaphore, queue và xây dựng các thành phần phục vụ hoạt động kiểm thử.

CHƯƠNG 4 – CÀI ĐẶT HỆ THỐNG

Sau khi hoàn thành quá trình thiết kế hệ thống ở Chương 3, chương này trình bày quá trình hiện thực hóa các thành phần của hệ thống kiểm thử trên nền tảng FreeRTOS Simulator. Nội dung tập trung vào việc xây dựng các task chức năng, cấu hình scheduler, triển khai cơ chế synchronization, giám sát deadline, thu thập dữ liệu runtime và mô phỏng các tình huống fault injection.

Mục tiêu của chương là hiện thực hóa kiến trúc đã thiết kế nhằm tạo ra môi trường kiểm thử có khả năng hỗ trợ các hoạt động Scheduler Testing, Priority Scheduling, Priority Preemption, Synchronization Testing, Deadline Testing, Fault Injection và Timing Analysis. Các thành phần được cài đặt theo hướng phục vụ trực tiếp cho quá trình thu thập dữ liệu thực nghiệm và đánh giá pass/fail ở các chương tiếp theo.

4.1 Môi trường cài đặt

Hệ thống được phát triển trên nền tảng FreeRTOS Windows Simulator nhằm tạo môi trường kiểm thử thuận tiện mà không phụ thuộc vào phần cứng nhúng thực tế. Môi trường này cho phép quan sát trực tiếp hoạt động của scheduler, trạng thái task, các sự kiện synchronization và dữ liệu runtime trong quá trình thực thi.

Bảng 4.1. Môi trường phát triển

Thành phần	Giá trị
Operating System	Windows 10/11
Programming Language	C
IDE	Visual Studio Code
Compiler	GCC
RTOS	FreeRTOS
Simulator	FreeRTOS Windows Simulator
Logging Format	Text Log
Version Control	Git

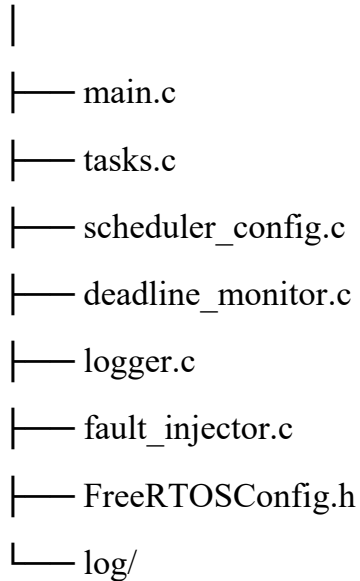
Việc sử dụng FreeRTOS Simulator giúp giảm thời gian phát triển, đồng thời hỗ trợ thu thập dữ liệu phục vụ các hoạt động Scheduler Testing, Priority Testing, Deadline Testing và Timing Analysis.

4.2 Cài đặt cấu trúc chương trình

Dựa trên kiến trúc đã trình bày trong Chương 3, hệ thống được tổ chức thành nhiều module độc lập nhằm thuận tiện cho việc quản lý và mở rộng.

Hình 4.1. Cấu trúc chương trình

Project



Trong đó, file main.c chịu trách nhiệm khởi tạo hệ thống, các task được định nghĩa trong tasks.c, module deadline_monitor.c thực hiện giám sát deadline, module logger.c thu thập dữ liệu runtime và fault_injector.c hỗ trợ các hoạt động fault injection.

4.3 Cài đặt Task và Priority

Các task được tạo thông qua API của FreeRTOS và được gán priority theo thiết kế đã trình bày ở Chương 3.

Bảng 4.2. Priority của các Task

Task	Priority
Emergency_Task	5
Control_Task	4
Deadline_Monitor_Task	4
Sensor_Task	3
Communication_Task	2
Fault_Injector_Task	2
Logger_Task	1

Ví dụ, Sensor_Task được khởi tạo bằng API của FreeRTOS như sau:

```
xTaskCreate(  
    Sensor_Task,
```

```

    "Sensor",
    STACK_SIZE,
    NULL,
    3,
    NULL
);

```

Tương tự, các task còn lại được tạo với mức priority tương ứng nhằm phục vụ các hoạt động Scheduler Testing và Priority Scheduling.

Emergency_Task được cấu hình với mức priority cao nhất nhằm đảm bảo scheduler thực hiện preemption ngay khi xuất hiện sự kiện khẩn cấp. Trong khi đó, Logger_Task được đặt ở mức priority thấp nhất để hạn chế ảnh hưởng tới các nhiệm vụ thời gian thực.

4.4 Cài đặt Synchronization

Các cơ chế synchronization được triển khai bằng mutex, semaphore và queue nhằm hỗ trợ trao đổi dữ liệu và bảo vệ tài nguyên dùng chung.

Bảng 4.3. Cơ chế Synchronization

Thành phần	Cơ chế
Sensor Data Queue	Queue
Communication Buffer	Queue
Shared Log Buffer	Mutex
System Status Variable	Mutex
Event Notification	Semaphore

Queue được sử dụng để truyền dữ liệu giữa Sensor_Task và Control_Task. Mutex được áp dụng cho các tài nguyên dùng chung như vùng log và biến trạng thái hệ thống. Semaphore được sử dụng để đồng bộ hóa các sự kiện giữa các task, đặc biệt trong các tình huống liên quan tới Emergency_Task và Deadline_Monitor_Task.

Những cơ chế này tạo nền tảng cho các test case Synchronization Testing và Concurrency Testing được trình bày trong chương tiếp theo.

4.5 Cài đặt Deadline Monitor, Logger và Fault Injector

Deadline_Monitor_Task được xây dựng nhằm giám sát khả năng đáp ứng deadline của các task trong hệ thống. Task này định kỳ kiểm tra thời gian thực thi và thời gian phản hồi của từng task, từ đó xác định các trường hợp deadline miss.

Logger_Task chịu trách nhiệm ghi nhận dữ liệu runtime phát sinh trong quá trình thực thi hệ thống. Các thông tin được lưu bao gồm thời điểm task bắt đầu, thời điểm hoàn thành, context switch, deadline miss và các sự kiện synchronization.

Bảng 4.4. Dữ liệu Runtime được thu thập

Dữ liệu	Mục đích
Execution Time	Timing Analysis
Response Time	Deadline Analysis
Latency	Performance Evaluation
WCET	Worst-Case Analysis
CPU Usage	Resource Analysis
Context Switch	Scheduler Evaluation
Deadline Miss	Pass/Fail Evaluation

Fault_Injector_Task được sử dụng để tạo các điều kiện bất lợi trong quá trình kiểm thử. Một số tình huống được triển khai bao gồm tăng tải CPU, trì hoãn task, chặn queue và tạo tranh chấp tài nguyên. Các tình huống này cho phép đánh giá độ ổn định của scheduler và khả năng đáp ứng của hệ thống trong điều kiện hoạt động không lý tưởng.

4.6 Tổng kết chương

Chương 4 đã trình bày quá trình cài đặt các thành phần chính của hệ thống kiểm thử trên nền tảng FreeRTOS Simulator. Trên cơ sở kiến trúc đã thiết kế ở Chương 3, nhóm đã hiện thực các task chức năng, cấu hình priority, triển khai các cơ chế synchronization, xây dựng deadline monitor, logger và fault injector.

Các thành phần sau khi cài đặt đã tạo thành một môi trường mô phỏng hoàn chỉnh, cho phép triển khai các hoạt động Scheduler Testing, Priority Scheduling, Priority Preemption, Synchronization Testing, Deadline Testing và Timing Analysis. Đồng thời, hệ thống cũng hỗ trợ thu thập dữ liệu runtime phục vụ cho việc đánh giá pass/fail và phân tích kết quả thực nghiệm.

Trên cơ sở hệ thống đã được cài đặt, Chương 5 sẽ trình bày quá trình thiết kế bộ kiểm thử, xây dựng các test suite và xác định tiêu chí đánh giá cho từng nhóm kiểm thử của đề tài.

CHƯƠNG 5 – THIẾT KẾ KIỂM THỬ

Sau khi hoàn thành việc thiết kế và cài đặt hệ thống FreeRTOS Simulator, bước tiếp theo là xây dựng bộ kiểm thử nhằm đánh giá hoạt động của scheduler, cơ chế synchronization, khả năng đáp ứng deadline và hiệu năng thời gian thực của hệ thống. Chương này trình bày chiến lược kiểm thử, tiêu chí đánh giá pass/fail và các test suite được xây dựng dựa trên các mục tiêu đã đề xuất trong proposal.

Các test case được thiết kế theo các kịch bản thực tế như Camera An Ninh Phát Hiện Xâm Nhập, Hệ Thống Báo Cháy Thông Minh, Drone Tránh Chướng Ngại Vật, Nhà Máy Có Emergency Stop và Trạm Quan Trắc Thời Tiết. Thông qua các kịch bản này, hệ thống được đánh giá dưới nhiều điều kiện hoạt động khác nhau nhằm xác minh tính đúng đắn của scheduler, khả năng xử lý đồng thời, cơ chế ưu tiên và mức độ đáp ứng thời gian thực.

5.1 Chiến lược kiểm thử

Dựa trên mục tiêu nghiên cứu đã trình bày trong Chương 1, các cơ sở lý thuyết ở Chương 2, kiến trúc hệ thống ở Chương 3 và quá trình cài đặt ở Chương 4, hoạt động kiểm thử được xây dựng theo hướng thực nghiệm kết hợp phân tích dữ liệu runtime. Mục tiêu của quá trình kiểm thử không chỉ dừng lại ở việc xác nhận hệ thống có thể thực thi thành công mà còn đánh giá mức độ đáp ứng của các cơ chế scheduler, priority scheduling, priority preemption, synchronization, concurrency, deadline handling và fault tolerance trong môi trường FreeRTOS Simulator.

Khác với các phương pháp kiểm thử chức năng thông thường, đề tài tập trung quan sát hành vi của hệ thống trong quá trình vận hành thời gian thực. Các task được cấu hình với mức priority, period, WCET và deadline khác nhau nhằm tạo ra các tình huống tương tự những hệ thống nhúng thực tế như Drone Tránh Chướng Ngại Vật, Camera An Ninh Phát Hiện Xâm Nhập, Hệ Thống Báo Cháy Thông Minh, Trạm Quan Trắc Thời Tiết và Nhà Máy Có Emergency Stop. Các kịch bản này cho phép đánh giá khả năng lập lịch, xử lý ưu tiên, phản ứng với sự kiện khẩn cấp, quản lý tài nguyên dùng chung và đáp ứng deadline của hệ thống.

Để đảm bảo tính hệ thống, các hoạt động kiểm thử được tổ chức thành nhiều test suite tương ứng với các mục tiêu của proposal, bao gồm Scheduler Testing, Priority Scheduling, Priority Preemption Testing, Hard Deadline Testing, Soft Deadline Testing, Synchronization Testing, Concurrency Testing, Fault Injection Testing và Timing Analysis Testing. Mỗi test suite bao gồm nhiều test case được thiết kế nhằm kiểm tra một hoặc nhiều cơ chế cụ thể của hệ thống. Việc phân chia thành các test suite giúp quá trình

đánh giá được thực hiện có cấu trúc, đồng thời thuận lợi cho việc so sánh kết quả giữa các nhóm kiểm thử.

Trong mỗi test case, nhóm xác định rõ mục tiêu kiểm thử, kịch bản thực hiện, các task tham gia, dữ liệu đầu vào, metrics cần thu thập, expected result và tiêu chí pass/fail. Dữ liệu runtime được ghi nhận thông qua Logger_Task và được sử dụng để tính toán các chỉ số như execution time, response time, latency, WCET, CPU usage, context switch count và deadline miss count. Các chỉ số này đóng vai trò là cơ sở định lượng để đánh giá hiệu năng và tính đúng đắn của hệ thống.

Thông qua chiến lược kiểm thử này, đề tài hướng tới việc đánh giá toàn diện hành vi của FreeRTOS Simulator trong nhiều điều kiện vận hành khác nhau. Kết quả thu được sẽ được sử dụng ở Chương 6 để phân tích khả năng đáp ứng thời gian thực, hiệu quả của scheduler, tính ổn định của cơ chế synchronization cũng như mức độ thành công của các phương pháp fault injection và timing analysis.

Hình 5.1. Tổng quan chiến lược kiểm thử

Proposal Objectives



Test Suite



Test Case



Runtime Logging



Metrics Collection



Pass / Fail Evaluation

5.2 Quy trình kiểm thử

Nhằm đảm bảo tính nhất quán giữa các lần chạy và tạo điều kiện cho việc so sánh kết quả giữa các nhóm kiểm thử, đề tài xây dựng một quy trình kiểm thử thống nhất cho toàn bộ test suite. Quy trình này được áp dụng cho các hoạt động Scheduler Testing, Priority Scheduling, Priority Preemption, Deadline Testing, Synchronization Testing,

Concurrency Testing, Fault Injection Testing và Timing Analysis Testing trong môi trường FreeRTOS Simulator.

Quá trình kiểm thử bắt đầu bằng việc lựa chọn test case và cấu hình các thành phần cần thiết cho kịch bản tương ứng. Tùy theo mục tiêu kiểm thử, hệ thống sẽ khởi tạo các task tham gia với priority, period, WCET và deadline đã được định nghĩa trong Chương 3. Đồng thời, các tài nguyên dùng chung như mutex, semaphore, queue và logger cũng được chuẩn bị trước khi scheduler bắt đầu hoạt động.

Sau khi hoàn thành bước khởi tạo, scheduler của FreeRTOS được kích hoạt để đưa hệ thống vào trạng thái thực thi. Trong quá trình chạy test case, các task tương tác với nhau thông qua queue, semaphore và mutex nhằm mô phỏng môi trường hoạt động của hệ thống nhúng đa tiến trình thời gian thực. Những sự kiện đặc biệt như task switching, preemption, deadline miss, queue blocking hoặc fault injection được tạo ra theo đúng kịch bản kiểm thử đã thiết kế.

Trong suốt quá trình vận hành, Logger_Task thực hiện ghi nhận dữ liệu runtime bao gồm timestamp, task name, trạng thái task, priority level, thời gian bắt đầu thực thi, thời gian hoàn thành, context switch và các sự kiện synchronization. Dữ liệu log này đóng vai trò là nguồn đầu vào cho hoạt động tính toán metrics và đánh giá hiệu năng hệ thống.

Sau khi test case kết thúc, dữ liệu runtime được xử lý để tính toán các chỉ số như execution time, response time, latency, WCET, CPU usage, context switch count và deadline miss count. Các giá trị thực tế thu được sẽ được so sánh với expected result và tiêu chí pass/fail đã xác định trước cho từng nhóm kiểm thử.

Thông qua quy trình kiểm thử thống nhất này, đề tài đảm bảo rằng mọi test case đều được thực hiện theo cùng một nguyên tắc, giúp kết quả thu được có tính lặp lại, dễ phân tích và phù hợp với mục tiêu đánh giá hệ thống nhúng đa tiến trình thời gian thực sử dụng FreeRTOS Simulator.

Hình 5.2. Quy trình thực hiện kiểm thử

Select Test Case



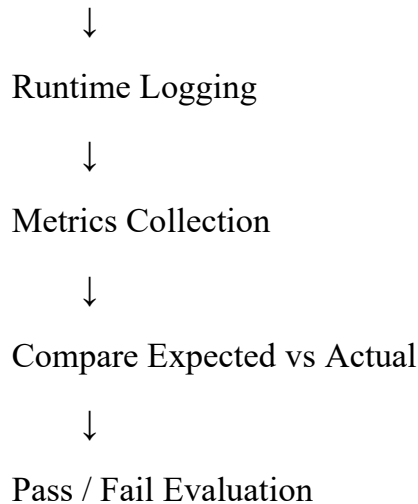
Initialize Tasks & Resources



Start FreeRTOS Scheduler



Run Test Scenario



Để minh họa cách hệ thống ghi nhận dữ liệu trong quá trình kiểm thử, cơ chế logging được triển khai thông qua các hàm ghi sự kiện runtime. Ví dụ, khi xảy ra sự kiện preemption hoặc deadline evaluation, hệ thống sẽ tạo log tương ứng nhằm phục vụ quá trình phân tích ở Chương 6.

```
log_event(  
    "Emergency_Task",  
    "PREEMPTION",  
    xTaskGetTickCount()  
);
```

Đoạn mã trên chỉ minh họa cách hệ thống ghi nhận sự kiện runtime trong quá trình chạy test case. Mã nguồn hoàn chỉnh của cơ chế logging và metrics collection được cung cấp trong file code đính kèm của đề tài.

5.3 Môi trường kiểm thử

Để phục vụ quá trình kiểm thử và đánh giá hệ thống, đề tài sử dụng môi trường FreeRTOS Windows Simulator nhằm mô phỏng hoạt động của hệ thống nhúng đa tiến trình thời gian thực trên máy tính. Việc sử dụng simulator cho phép nhóm quan sát trực tiếp hành vi của scheduler, cơ chế synchronization, quá trình xử lý deadline và các dữ liệu runtime mà không cần triển khai trên phần cứng nhúng thực tế.

Môi trường kiểm thử được cấu hình theo các thông số phù hợp với yêu cầu của đề tài. Toàn bộ các task được triển khai bằng ngôn ngữ C và thực thi dưới sự quản lý của FreeRTOS Scheduler. Trong quá trình kiểm thử, hệ thống sử dụng cơ chế runtime logging

để ghi nhận dữ liệu thực thi, đồng thời thu thập các metrics phục vụ hoạt động timing analysis và pass/fail evaluation.

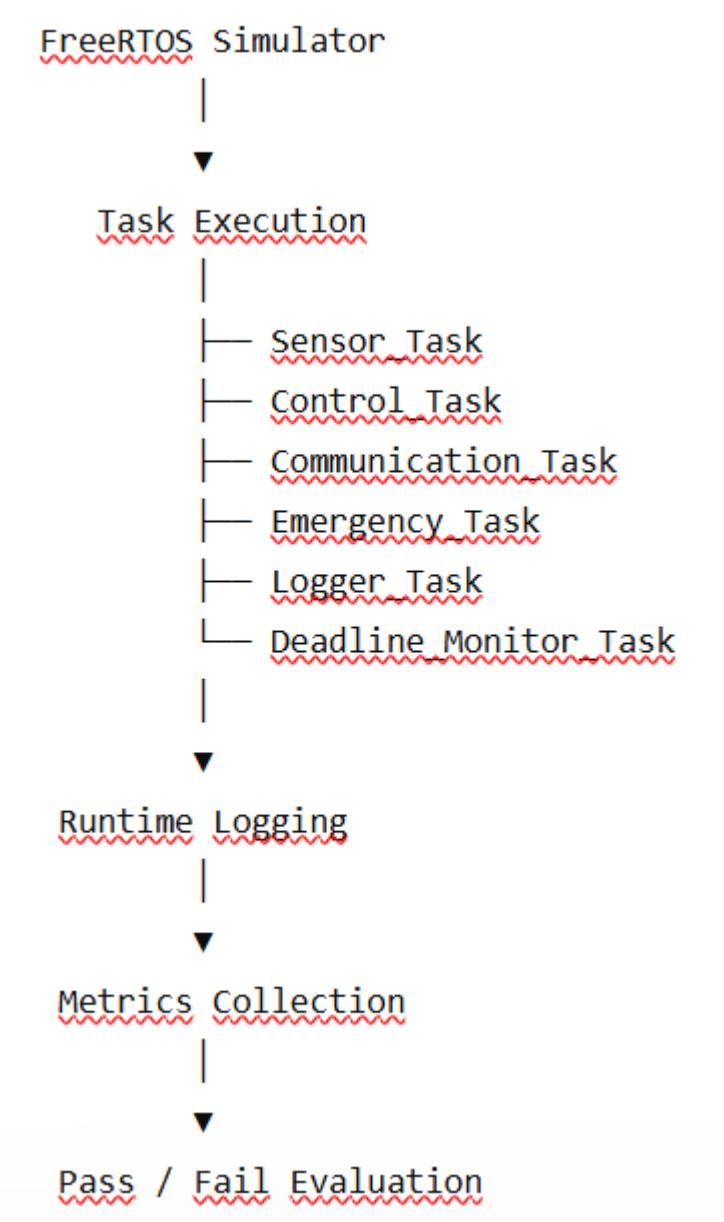
Bảng 5.1. Môi trường kiểm thử

Thành phần	Cấu hình
Nền tảng kiểm thử	FreeRTOS Windows Simulator
Ngôn ngữ lập trình	C
Hệ điều hành	Windows 10/11
Scheduler	Priority-Based Preemptive Scheduler
Tick Rate	1 ms
Logging	Runtime Text Log
Metrics Collection	Execution Time, Response Time, Latency, WCET, CPU Usage, Context Switch, Deadline Miss
Công cụ phát triển	Visual Studio Code
Compiler	GCC

Trong quá trình chạy test case, các task được cấu hình với priority, period, WCET và deadline khác nhau nhằm mô phỏng các tình huống thường gặp trong hệ thống nhúng thực tế. Các sự kiện như task switching, priority preemption, synchronization, deadline miss và fault injection đều được ghi nhận vào log runtime để phục vụ quá trình phân tích ở Chương 6.

Việc sử dụng FreeRTOS Simulator mang lại lợi thế trong giai đoạn nghiên cứu và kiểm thử bởi nhóm có thể nhanh chóng thay đổi cấu hình task, tạo các tình huống fault injection và thu thập dữ liệu thực nghiệm mà không bị phụ thuộc vào đặc tính của phần cứng. Đồng thời, môi trường này vẫn duy trì đầy đủ các cơ chế cốt lõi của FreeRTOS như scheduler, queue, mutex, semaphore và task management, đảm bảo tính phù hợp đối với mục tiêu đánh giá hệ thống nhúng đa tiến trình thời gian thực của đề tài.

Hình 5.3. Môi trường kiểm thử của hệ thống



5.4 Runtime Metrics và Tiêu chí Pass/Fail

Nhằm đảm bảo quá trình kiểm thử có tính định lượng và phù hợp với đặc trưng của hệ thống nhúng thời gian thực, đề tài sử dụng tập hợp các runtime metrics để đánh giá hành vi của hệ thống trong quá trình thực thi. Các chỉ số này được thu thập thông qua Logger_Task và Deadline_Monitor_Task trong môi trường FreeRTOS Simulator, đóng vai trò làm cơ sở cho các hoạt động Scheduler Testing, Priority Testing, Synchronization Testing, Deadline Testing, Fault Injection và Timing Analysis.

Khác với các phương pháp đánh giá dựa trên kết quả đầu ra đơn thuần, hệ thống của đề tài tập trung quan sát các thông số phản ánh trực tiếp hoạt động của scheduler, mức

độ đáp ứng deadline, hiệu quả synchronization và mức độ ổn định của hệ thống khi nhiều task cùng hoạt động đồng thời.

Bảng 5.2. Runtime Metrics sử dụng trong kiểm thử

Metric	Mục đích sử dụng
Execution Time	Đánh giá thời gian thực thi của task
Response Time	Đánh giá khả năng phản hồi của hệ thống
Latency	Đo độ trễ xử lý giữa các sự kiện
WCET	Phân tích thời gian thực thi lớn nhất của task
CPU Usage	Đánh giá mức sử dụng CPU
Context Switch Count	Phân tích hoạt động của scheduler
Deadline Miss Count	Đánh giá khả năng đáp ứng deadline
Queue Length	Theo dõi trạng thái hàng đợi dữ liệu
Semaphore Wait Time	Đánh giá hiệu quả synchronization

Execution Time được sử dụng để xác định khoảng thời gian một task cần để hoàn thành xử lý. Response Time phản ánh thời gian từ khi task chuyển sang trạng thái Ready đến khi thực sự được scheduler cấp CPU. Latency được dùng để đánh giá độ trễ phát sinh trong quá trình trao đổi dữ liệu hoặc xử lý sự kiện. Đối với các nhiệm vụ thời gian thực, Worst-Case Execution Time (WCET) đóng vai trò quan trọng trong việc đánh giá giới hạn thời gian thực thi lớn nhất của task trong điều kiện tải cao hoặc khi xảy ra preemption.

Bên cạnh các metrics thời gian, hệ thống còn ghi nhận CPU Usage, Context Switch Count, Queue Length và Semaphore Wait Time nhằm phục vụ việc đánh giá scheduler, synchronization và concurrency. Những chỉ số này đặc biệt hữu ích trong các kịch bản fault injection, nơi hệ thống phải hoạt động dưới điều kiện CPU overload, queue blocking hoặc resource contention.

Để đảm bảo tính khách quan của quá trình kiểm thử, đề tài xây dựng tiêu chí pass/fail riêng cho từng nhóm test suite. Các tiêu chí này được xác định dựa trên expected behavior của scheduler, deadline handling, synchronization và timing analysis.

Bảng 5.3. Tiêu chí Pass/Fail cho các nhóm kiểm thử

Nhóm kiểm thử	PASS Criteria	FAIL Criteria
Scheduler Testing	Task thực thi đúng thứ tự priority và trạng thái	Task chạy sai thứ tự hoặc không được scheduler cấp CPU
Priority Scheduling	Task priority cao thực thi trước task priority thấp	Task priority thấp chạy trước task priority cao

Priority Preemption	Emergency_Task preempt task đang chạy đúng thời điểm	Không xảy ra preemption hoặc response time vượt deadline
Hard Deadline Testing	Không xuất hiện deadline miss	Có deadline miss
Soft Deadline Testing	Task trễ trong ngưỡng cho phép	Trễ vượt mức cho phép
Synchronization Testing	Không xảy ra race condition hoặc sai lệch dữ liệu	Race condition hoặc dữ liệu không nhất quán
Concurrency Testing	Hệ thống duy trì hoạt động ổn định	Deadlock, starvation hoặc crash
Fault Injection Testing	Hệ thống phát hiện hoặc chịu được lỗi mô phỏng	Hệ thống treo hoặc mất khả năng phản hồi
Timing Analysis Testing	Metrics nằm trong ngưỡng thiết kế	Metrics vượt threshold quy định

Để tăng tính traceability giữa mục tiêu của proposal, nhóm test suite và các chỉ số đánh giá runtime, đề tài xây dựng Master Test Matrix nhằm ánh xạ giữa objective nghiên cứu, hoạt động kiểm thử, metrics thu thập và tiêu chí pass/fail. Bảng này giúp đảm bảo mọi mục tiêu của đề tài đều được kiểm chứng thông qua các dữ liệu định lượng cụ thể.

Bảng 5.4. Master Test Matrix cho hoạt động kiểm thử

Objective	Test Suite	Metrics	Pass Criteria
Scheduler Evaluation	Scheduler Testing	Execution Order, Context Switch	Correct Task Execution Order
Priority Scheduling	Priority Scheduling Testing	Response Time	Higher Priority Task Executes First
Priority Preemption	Priority Preemption Testing	Response Time, Preemption Time	Emergency_Task Preempts Successfully
Hard Deadline Evaluation	Hard Deadline Testing	Deadline Miss, WCET	No Deadline Miss
Soft Deadline Evaluation	Soft Deadline Testing	Latency, Response Time	Delay Within Allowed Threshold
Synchronization Evaluation	Synchronization Testing	Semaphore Wait Time, Queue Length	No Race Condition

Concurrency Evaluation	Concurrency Testing	CPU Usage, Context Switch	Stable Multi-Task Operation
Fault Injection Evaluation	Fault Injection Testing	CPU Usage, Latency	System Remains Operational
Timing Analysis	Timing Analysis Testing	WCET, Execution Time, Response Time	Metrics Within Design Limits

Thông qua Master Test Matrix, các nhóm kiểm thử được liên kết trực tiếp với mục tiêu của đề tài, đồng thời tạo cơ sở thống nhất cho việc thu thập dữ liệu runtime, đánh giá expected behavior và xác định trạng thái pass/fail của từng test case.

Trong quá trình đánh giá, hệ thống sử dụng dữ liệu runtime thu thập từ log để so sánh giữa expected result và actual result của từng test case. Chẳng hạn, trong Priority Preemption Testing, Emergency_Task phải được scheduler cấp CPU ngay sau khi chuyển sang trạng thái Ready và response time phải nhỏ hơn deadline quy định. Đối với Hard Deadline Testing, chỉ cần xuất hiện một deadline miss cũng đủ để xác định test case ở trạng thái fail.

Ví dụ, việc đánh giá deadline trong hệ thống được thực hiện dựa trên các điều kiện logic như sau:

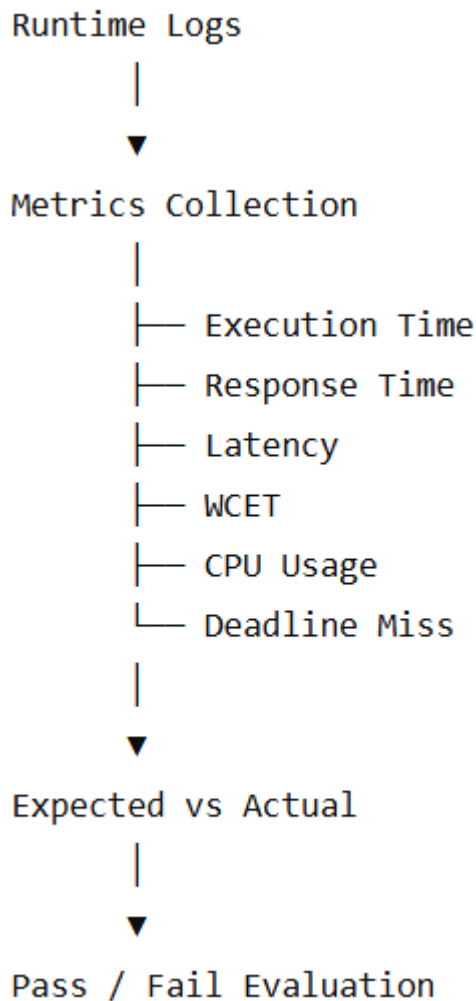
```

if(response_time <= deadline)
{
    result = PASS;
}
else
{
    result = FAIL;
}

```

Đoạn mã trên minh họa cách hệ thống xác định trạng thái pass/fail dựa trên response time và deadline của task. Mã nguồn hoàn chỉnh của cơ chế deadline monitoring và metrics collection được trình bày trong file code đính kèm của đề tài.

Hình 5.4. Quan hệ giữa Runtime Metrics và Pass/Fail Evaluation



5.5 Scheduler Testing

Scheduler đóng vai trò trung tâm trong hệ thống FreeRTOS bởi đây là thành phần chịu trách nhiệm lựa chọn task được cấp CPU tại từng thời điểm. Trong hệ thống của đề tài, scheduler phải quản lý đồng thời nhiều task với mức priority, chu kỳ thực thi và deadline khác nhau. Do đó, việc kiểm thử scheduler trở thành bước cần thiết nhằm xác nhận tính đúng đắn của cơ chế lập lịch trước khi đánh giá các chức năng nâng cao như preemption, synchronization hay deadline handling.

Nhóm kiểm thử Scheduler Testing được thiết kế nhằm đánh giá khả năng điều phối task của FreeRTOS Scheduler trong các điều kiện hoạt động khác nhau. Các test case tập trung vào việc kiểm tra execution order, response time, trạng thái task và khả năng duy trì chu kỳ thực thi của các nhiệm vụ định kỳ.

Bảng 5.4. Scheduler Testing Summary

Mã Test Case	Kịch bản kiểm thử	Metrics chính	Pass Criteria
SCH-01	Sensor Kiểm Soát Người Ra Vào	Execution Order, Response Time	Task chạy đúng period
SCH-02	Trạm Quan Trắc Thời Tiết	Execution Time, Jitter	Scheduler duy trì chu kỳ
SCH-03	Logger Hoạt Động Nền	CPU Usage, Context Switch	Task nền không ảnh hưởng task chính

5.5.1 SCH-01 — Sensor Kiểm Soát Người Ra Vào

Test case này mô phỏng hệ thống kiểm soát người ra vào sử dụng cảm biến nhận diện và xử lý dữ liệu theo chu kỳ cố định. Mục tiêu của test case là đánh giá khả năng scheduler quản lý các task định kỳ trong môi trường nhiều nhiệm vụ cùng hoạt động.

Trong kịch bản này, Sensor_Task tạo dữ liệu đầu vào theo chu kỳ 100 ms và gửi dữ liệu tới Control_Task thông qua queue. Đồng thời, Logger_Task tiếp tục hoạt động ở mức priority thấp hơn để ghi nhận dữ liệu runtime. Scheduler phải đảm bảo các task được thực thi theo đúng chu kỳ thiết kế mà không gây sai lệch execution order.

Bảng 5.5. SCH-01 Configuration

Thuộc tính	Giá trị
Test Suite	Scheduler Testing
Kịch bản	Sensor Kiểm Soát Người Ra Vào
Task tham gia	Sensor_Task, Control_Task, Logger_Task
Metrics	Execution Order, Response Time
Expected Result	Task thực thi đúng chu kỳ
Pass Criteria	Response Time trong ngưỡng thiết kế

Trong quá trình kiểm thử, dữ liệu runtime được thu thập nhằm xác định thời điểm task chuyển trạng thái Ready, thời điểm được scheduler cấp CPU và thời gian hoàn thành xử lý. Test case đạt trạng thái pass nếu scheduler duy trì đúng period của Sensor_Task và không xuất hiện sai lệch execution order.

5.5.2 SCH-02 — Trạm Quan Trắc Thời Tiết

Test case này được xây dựng nhằm đánh giá khả năng duy trì hoạt động của scheduler khi hệ thống phải xử lý nhiều task định kỳ với chu kỳ khác nhau.

Trong kịch bản Trạm Quan Trắc Thời Tiết, Sensor_Task thực hiện thu thập dữ liệu môi trường theo chu kỳ cố định, Communication_Task truyền dữ liệu ra bên ngoài và

Logger_Task tiếp tục hoạt động nền. Scheduler phải phân bổ CPU hợp lý để các task đều được thực thi đúng chu kỳ mà không gây tích lũy độ trễ.

Bảng 5.6. SCH-02 Configuration

Thuộc tính	Giá trị
Test Suite	Scheduler Testing
Kịch bản	Trạm Quan Trắc Thời Tiết
Task tham gia	Sensor_Task, Communication_Task, Logger_Task
Metrics	Execution Time, Jitter
Expected Result	Scheduler duy trì period ổn định
Pass Criteria	Jitter trong ngưỡng cho phép

Test case tập trung phân tích khả năng duy trì period và mức độ ổn định của scheduler trong điều kiện tải trung bình. Dữ liệu runtime được sử dụng để đánh giá execution time, khoảng cách giữa các lần chạy task và mức độ dao động chu kỳ (jitter).

5.5.3 SCH-03 — Logger Hoạt Động Nền

Mục tiêu của test case này là kiểm tra ảnh hưởng của các task nền có priority thấp tới scheduler và các nhiệm vụ quan trọng hơn trong hệ thống.

Trong kịch bản này, Logger_Task liên tục ghi nhận dữ liệu runtime trong khi Sensor_Task và Control_Task thực hiện các nhiệm vụ thời gian thực. Scheduler phải đảm bảo task nền không chiếm dụng CPU vượt mức cho phép và không làm ảnh hưởng tới response time của các task chính.

Bảng 5.7. SCH-03 Configuration

Thuộc tính	Giá trị
Test Suite	Scheduler Testing
Kịch bản	Logger Hoạt Động Nền
Task tham gia	Logger_Task, Sensor_Task, Control_Task
Metrics	CPU Usage, Context Switch Count
Expected Result	Task nền không ảnh hưởng hệ thống
Pass Criteria	CPU Usage nằm trong ngưỡng thiết kế

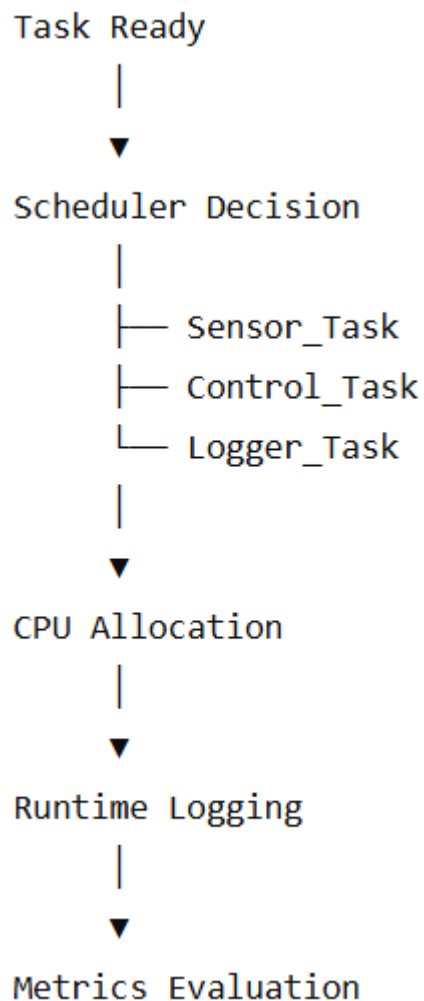
Để hỗ trợ quá trình đánh giá, hệ thống sử dụng log runtime nhằm ghi nhận execution order, thời điểm task switching và số lần context switch phát sinh trong quá trình chạy test case.

Ví dụ, hệ thống sử dụng cơ chế logging để ghi nhận hoạt động của scheduler như sau:


```
log_event(  
    "Scheduler",  
    "TASK_SWITCH",  
    xTaskGetTickCount()  
);
```

Đoạn mã trên minh họa cách hệ thống ghi nhận các sự kiện task switching trong quá trình kiểm thử scheduler. Mã nguồn hoàn chỉnh được cung cấp trong file code của đề tài.

Hình 5.5. Scheduler Testing Workflow



Thông qua nhóm Scheduler Testing, đề tài đánh giá khả năng điều phối task của FreeRTOS Scheduler trong môi trường đa nhiệm, đồng thời tạo nền tảng cho các hoạt

động kiểm thử priority scheduling, preemption và deadline handling được trình bày ở các mục tiếp theo.

5.6 Priority Scheduling và Priority Preemption Testing

Dựa trên cơ chế scheduler và priority model đã thiết kế trong Chương 3, nhóm xây dựng bộ kiểm thử Priority Scheduling và Priority Preemption nhằm đánh giá khả năng phân bổ CPU theo mức ưu tiên và phản ứng của hệ thống trước các sự kiện khẩn cấp.

Các test case trong nhóm này tập trung kiểm tra hành vi của scheduler khi nhiều task cùng ở trạng thái Ready, đồng thời đánh giá khả năng preemption của Emergency_Task trong các tình huống yêu cầu phản hồi thời gian thực. Trong hệ thống FreeRTOS Simulator của đề tài, mỗi task được gán một mức priority riêng. Emergency_Task được cấu hình với priority cao nhất nhằm mô phỏng các tình huống khẩn cấp như phát hiện cháy, phát hiện xâm nhập hoặc dừng khẩn cấp trong nhà máy. Nhóm kiểm thử này được xây dựng nhằm đánh giá khả năng scheduler lựa chọn task có priority cao nhất và khả năng preempt các task đang chạy khi xuất hiện nhiệm vụ khẩn cấp.

Bảng 5.8. Priority Scheduling & Preemption Testing Summary

Mã Test Case	Kịch bản kiểm thử	Metrics chính	Pass Criteria
PRI-01	Camera An Ninh Phát Hiện Xâm Nhập	Response Time, Context Switch	Emergency_Task được ưu tiên
PRI-02	Drone Tránh Chướng Ngại Vật	Response Time, Deadline Miss	Task điều khiển chạy trước
PRE-01	Nhà Máy Có Emergency Stop	Response Time, Preemption Time	Preemption xảy ra ngay

5.6.1 PRI-01 — Camera An Ninh Phát Hiện Xâm Nhập

Test case này mô phỏng hệ thống camera an ninh đang thực hiện ghi log và truyền dữ liệu định kỳ thì phát hiện tín hiệu xâm nhập. Khi đó, Emergency_Task được kích hoạt với priority cao nhất nhằm xử lý sự kiện khẩn cấp.

Mục tiêu của test case là kiểm tra khả năng priority scheduling của FreeRTOS Scheduler khi đồng thời tồn tại nhiều task ở trạng thái Ready. Scheduler phải lựa chọn Emergency_Task thay vì các task có priority thấp hơn.

Bảng 5.9. PRI-01 Configuration

Thuộc tính	Giá trị
------------	---------

Test Suite	Priority Scheduling
Kịch bản	Camera An Ninh Phát Hiện Xâm Nhập
Task tham gia	Sensor_Task, Emergency_Task, Logger_Task
Metrics	Response Time, Context Switch Count
Expected Result	Emergency_Task được thực thi đầu tiên
Pass Criteria	Response Time ≤ 20 ms

Trong quá trình kiểm thử, hệ thống ghi nhận thời điểm phát hiện xâm nhập và thời điểm Emergency_Task bắt đầu thực thi. Dữ liệu này được sử dụng để đánh giá khả năng phản hồi của scheduler đối với các nhiệm vụ ưu tiên cao.

5.6.2 PRI-02 — Drone Tránh Chướng Ngại Vật

Kịch bản này mô phỏng drone đang thực hiện di chuyển và liên tục nhận dữ liệu từ cảm biến khoảng cách. Khi phát hiện vật cản ở khoảng cách nguy hiểm, Control_Task phải được scheduler ưu tiên xử lý nhằm đưa ra lệnh tránh va chạm.

Mục tiêu của test case là đánh giá khả năng priority scheduling trong môi trường nhiều task hoạt động đồng thời. Scheduler phải đảm bảo các task điều khiển có mức ưu tiên cao hơn được cấp CPU trước các task phụ trợ như logging hoặc communication.

Bảng 5.10. PRI-02 Configuration

Thuộc tính	Giá trị
Test Suite	Priority Scheduling
Kịch bản	Drone Tránh Chướng Ngại Vật
Task tham gia	Sensor_Task, Control_Task, Logger_Task
Metrics	Response Time, Deadline Miss
Expected Result	Control_Task được ưu tiên thực thi
Pass Criteria	Không xuất hiện deadline miss

Test case đạt trạng thái pass khi Control_Task luôn được thực thi đúng thời điểm và hoàn thành xử lý trước deadline quy định.

5.6.3 PRE-01 — Nhà Máy Có Emergency Stop

Đây là test case quan trọng nhất trong nhóm Priority Preemption Testing. Hệ thống mô phỏng môi trường sản xuất đang vận hành bình thường thì xuất hiện tín hiệu Emergency Stop yêu cầu dừng toàn bộ hoạt động.

Tại thời điểm sự kiện khẩn cấp xuất hiện, Emergency_Task chuyển sang trạng thái Ready với priority cao nhất. Scheduler phải ngay lập tức thực hiện preemption, tạm dừng task đang chạy và chuyển CPU cho Emergency_Task.

Bảng 5.11. PRE-01 Configuration

Thuộc tính	Giá trị
Test Suite	Priority Preemption
Kịch bản	Nhà Máy Có Emergency Stop
Task tham gia	Control_Task, Logger_Task, Emergency_Task
Metrics	Response Time, Preemption Time
Expected Result	Emergency_Task preempt ngay lập tức
Pass Criteria	Response Time ≤ 20 ms và không deadline miss

Test case này cho phép đánh giá trực tiếp hiệu quả của cơ chế preemption trong FreeRTOS. Các dữ liệu runtime được sử dụng để xác định khoảng thời gian từ lúc Emergency_Task chuyển sang trạng thái Ready cho tới khi bắt đầu thực thi.

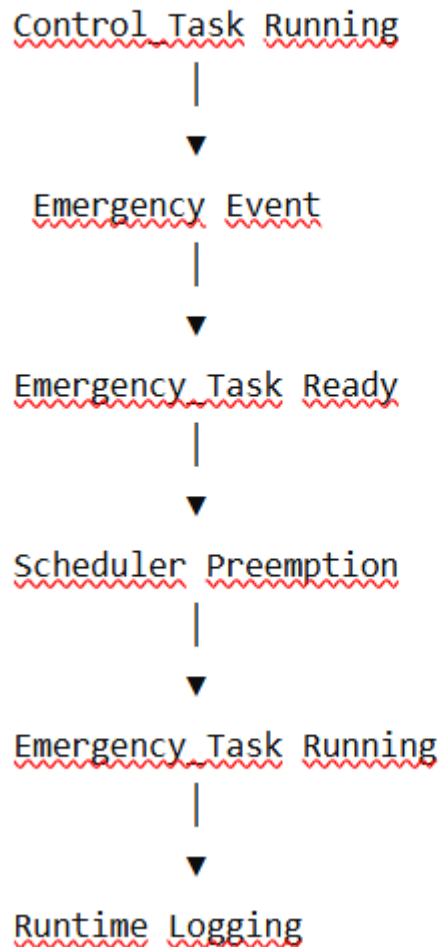
Để hỗ trợ hoạt động kiểm thử, hệ thống sử dụng cơ chế logging nhằm ghi nhận thời điểm xảy ra preemption và chuyển đổi task:

```
log_event(
    "Emergency_Task",
    "PREEMPTION",
    xTaskGetTickCount()
);
```

Đoạn mã trên minh họa cách hệ thống ghi nhận sự kiện preemption trong quá trình kiểm thử. Dữ liệu log thu được sẽ được sử dụng để tính toán response time và đánh giá khả năng đáp ứng của scheduler trong Chương 6.

Hình 5.6. Priority Preemption Workflow

Hình 5.6. Priority Preemption Workflow



Thông qua nhóm kiểm thử Priority Scheduling và Priority Preemption, đề tài đánh giá khả năng ưu tiên các nhiệm vụ quan trọng và phản ứng của scheduler trước các sự kiện khẩn cấp. Đây là cơ sở để tiếp tục đánh giá khả năng đáp ứng deadline trong mục tiếp theo.

5.7 Hard Deadline và Soft Deadline Testing

Dựa trên deadline model đã xây dựng trong Chương 3, nhóm triển khai bộ kiểm thử Hard Deadline và Soft Deadline nhằm đánh giá khả năng đáp ứng thời gian thực của hệ thống trong các điều kiện hoạt động khác nhau.

Thông qua các kịch bản báo cháy, emergency stop và quan trắc môi trường, nhóm phân tích khả năng đáp ứng deadline, phát hiện deadline miss và đánh giá các chỉ số timing metrics thu được từ FreeRTOS Simulator. Trong hệ thống FreeRTOS Simulator

của đề tài, các task được cấu hình với period, WCET và deadline khác nhau nhằm mô phỏng cả hard deadline và soft deadline.

Deadline_Monitor_Task chịu trách nhiệm giám sát quá trình thực thi của các task và ghi nhận các trường hợp deadline miss. Nhóm kiểm thử này được xây dựng nhằm đánh giá khả năng đáp ứng deadline của scheduler trong nhiều điều kiện hoạt động khác nhau.

Bảng 5.12. Hard Deadline và Soft Deadline Testing Summary

Mã Test Case	Kịch bản kiểm thử	Loại Deadline	Metrics chính
HD-01	Hệ Thống Báo Cháy Thông Minh	Hard Deadline	Response Time, Deadline Miss
HD-02	Nhà Máy Có Emergency Stop	Hard Deadline	WCET, Deadline Miss
SD-01	Trạm Quan Trắc Thời Tiết	Soft Deadline	Response Time, Latency
SD-02	Hệ Thống Giám Sát Chất Lượng Không Khí	Soft Deadline	Latency, CPU Usage

5.7.1 HD-01 — Hệ Thống Báo Cháy Thông Minh

Kịch bản này mô phỏng hệ thống báo cháy sử dụng cảm biến nhiệt độ và khói để phát hiện nguy cơ cháy nổ. Khi tín hiệu bất thường xuất hiện, Emergency_Task phải được kích hoạt và hoàn thành xử lý trong khoảng thời gian quy định.

Mục tiêu của test case là đánh giá khả năng đáp ứng hard deadline của hệ thống. Scheduler phải đảm bảo Emergency_Task luôn được thực thi trước deadline bất kể trạng thái của các task khác.

Bảng 5.13. HD-01 Configuration

Thuộc tính	Giá trị
Test Suite	Hard Deadline Testing
Kịch bản	Hệ Thống Báo Cháy Thông Minh
Task tham gia	Sensor_Task, Emergency_Task, Deadline_Monitor_Task
Deadline	20 ms
Metrics	Response Time, Deadline Miss Count
Expected Result	Không xuất hiện deadline miss
Pass Criteria	Deadline Miss Count = 0

Trong quá trình kiểm thử, `Deadline_Monitor_Task` ghi nhận thời điểm `Emergency_Task` chuyển sang trạng thái Ready và thời điểm hoàn thành xử lý. Nếu thời gian phản hồi hoặc thời gian thực thi vượt quá deadline đã cấu hình, test case sẽ được đánh giá là fail.

5.7.2 HD-02 — Nhà Máy Có Emergency Stop

Test case này mô phỏng hệ thống điều khiển trong môi trường sản xuất công nghiệp. Khi xuất hiện tín hiệu Emergency Stop, hệ thống phải dừng hoạt động của các thiết bị ngay lập tức nhằm đảm bảo an toàn vận hành.

Mục tiêu của test case là đánh giá khả năng đáp ứng hard deadline trong điều kiện hệ thống đang chịu tải. Scheduler phải đảm bảo `Emergency_Task` được thực thi trong thời gian ngắn nhất và hoàn thành trước deadline quy định.

Bảng 5.14. HD-02 Configuration

Thuộc tính	Giá trị
Test Suite	Hard Deadline Testing
Kịch bản	Nhà Máy Có Emergency Stop
Task tham gia	<code>Control_Task</code> , <code>Emergency_Task</code> , <code>Deadline_Monitor_Task</code>
Deadline	20 ms
Metrics	WCET, Deadline Miss Count
Expected Result	Không vi phạm deadline
Pass Criteria	$WCET \leq \text{Deadline}$

Bên cạnh response time, test case còn tập trung đánh giá Worst-Case Execution Time (WCET) nhằm xác định thời gian thực thi lớn nhất của `Emergency_Task` trong điều kiện có nhiều task hoạt động đồng thời.

5.7.3 SD-01 — Trạm Quan Trắc Thời Tiết

Khác với các nhiệm vụ an toàn, việc gửi dữ liệu môi trường trong hệ thống quan trắc thời tiết có thể chấp nhận một mức độ trễ nhất định mà không làm mất ý nghĩa của dữ liệu.

Test case này được thiết kế nhằm đánh giá khả năng xử lý soft deadline của hệ thống. Scheduler vẫn phải đảm bảo task được thực thi định kỳ, tuy nhiên một số độ trễ nhỏ được xem là chấp nhận được.

Bảng 5.15. SD-01 Configuration

Thuộc tính	Giá trị
Test Suite	Soft Deadline Testing
Kịch bản	Trạm Quan Trắc Thời Tiết

Task tham gia	Sensor_Task, Communication_Task
Deadline	150 ms
Metrics	Response Time, Latency
Expected Result	Task hoàn thành gần deadline
Pass Criteria	Latency trong ngưỡng cho phép

Trong quá trình đánh giá, hệ thống tập trung phân tích latency và response time thay vì yêu cầu tuyệt đối không được xuất hiện độ trễ.

5.7.4 SD-02 — Hệ Thống Giám Sát Chất Lượng Không Khí

Kịch bản này mô phỏng hệ thống giám sát môi trường thực hiện thu thập và truyền dữ liệu chất lượng không khí định kỳ. Các dữ liệu này phục vụ mục đích theo dõi dài hạn nên có thể chấp nhận một số sai lệch nhỏ về thời gian.

Bảng 5.16. SD-02 Configuration

Thuộc tính	Giá trị
Test Suite	Soft Deadline Testing
Kịch bản	Hệ Thống Giám Sát Chất Lượng Không Khí
Task tham gia	Sensor_Task, Communication_Task, Logger_Task
Deadline	200 ms
Metrics	Latency, CPU Usage
Expected Result	Hệ thống duy trì hoạt động ổn định
Pass Criteria	CPU Usage và Latency trong ngưỡng thiết kế

Test case này cho phép đánh giá ảnh hưởng của tải hệ thống tới các nhiệm vụ soft deadline và khả năng duy trì hoạt động ổn định khi số lượng task tăng lên.

Để hỗ trợ đánh giá deadline, hệ thống sử dụng cơ chế giám sát thời gian thực thi như sau:

```
if(response_time <= deadline)
```

```
{
```

```
    result = PASS;
```

```
}
```

```
else
```

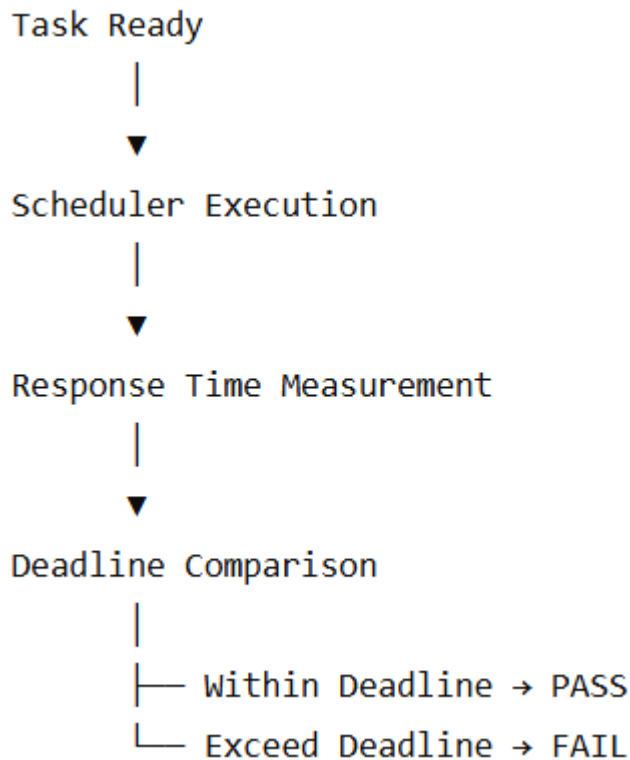
```
{
```



```
    result = FAIL;
}
```

Đoạn mã trên minh họa cách Deadline_Monitor_Task xác định trạng thái pass/fail dựa trên response time và deadline của task.

Hình 5.7. Deadline Evaluation Workflow



Thông qua nhóm kiểm thử Hard Deadline và Soft Deadline, đề tài đánh giá khả năng đáp ứng thời gian thực của FreeRTOS Scheduler trong nhiều điều kiện vận hành khác nhau. Kết quả thu được sẽ được sử dụng ở Chương 6 để phân tích deadline miss, response time, WCET và mức độ ổn định của hệ thống.

5.8 Synchronization và Concurrency Testing

Để đánh giá hiệu quả của các cơ chế synchronization đã thiết kế trong Chương 3, nhóm xây dựng bộ kiểm thử Synchronization và Concurrency Testing dựa trên các tình huống nhiều task cùng truy cập tài nguyên dùng chung.

Các test case tập trung kiểm tra khả năng bảo vệ Shared Log Buffer, quản lý Queue Communication và duy trì hoạt động ổn định của hệ thống khi nhiều task thực thi đồng thời.

Trong hệ thống FreeRTOS Simulator của đề tài, các cơ chế mutex, semaphore và queue được sử dụng để quản lý việc truy cập tài nguyên dùng chung giữa các task. Nhóm kiểm thử Synchronization và Concurrency Testing được xây dựng nhằm đánh giá tính đúng đắn của các cơ chế đồng bộ hóa cũng như khả năng duy trì hoạt động ổn định khi nhiều task thực thi đồng thời.

Bảng 5.17. Synchronization và Concurrency Testing Summary

Mã Test Case	Kịch bản	Cơ chế
SYN-01	Hệ Thống Ghi Log Nhà Thông Minh	Mutex
SYN-02	Camera An Ninh Ghi Log Đồng Thời	Queue
CON-01	Drone Xử Lý Nhiều Cảm Biến	Concurrency
CON-02	Nhà Máy Nhiều Tín Hiệu Điều Khiển	Shared Resource

5.8.1 SYN-01 — Hệ Thống Ghi Log Nhà Thông Minh

Kịch bản này mô phỏng hệ thống nhà thông minh trong đó nhiều task cùng ghi dữ liệu vào Shared Log Buffer. Logger_Task, Sensor_Task và Communication_Task đều có nhu cầu truy cập tài nguyên này trong quá trình hoạt động.

Mục tiêu của test case là đánh giá khả năng bảo vệ tài nguyên dùng chung bằng mutex và xác minh rằng dữ liệu log không bị sai lệch khi nhiều task truy cập đồng thời.

Bảng 5.18. SYN-01 Configuration

Thuộc tính	Giá trị
Test Suite	Synchronization Testing
Kịch bản	Hệ Thống Ghi Log Nhà Thông Minh
Shared Resource	Shared Log Buffer
Cơ chế sử dụng	Mutex
Metrics	Semaphore Wait Time, Context Switch Count
Expected Result	Không xuất hiện race condition
Pass Criteria	Dữ liệu log nhất quán

Trong quá trình kiểm thử, hệ thống ghi nhận thời gian chờ mutex, số lần context switch và tính toàn vẹn của dữ liệu log nhằm đánh giá hiệu quả của cơ chế synchronization.

5.8.2 SYN-02 — Camera An Ninh Ghi Log Đồng Thời

Test case này mô phỏng tình huống Camera_Task và Emergency_Task cùng tạo dữ liệu cần ghi nhận vào hệ thống log. Dữ liệu được truyền thông qua queue trước khi được Logger_Task xử lý.

Mục tiêu của test case là đánh giá khả năng trao đổi dữ liệu giữa các task thông qua queue và kiểm tra hiện tượng queue overflow hoặc mất dữ liệu.

Bảng 5.19. SYN-02 Configuration

Thuộc tính	Giá trị
Test Suite	Synchronization Testing
Kịch bản	Camera An Ninh Ghi Log Đồng Thời
Shared Resource	Log Queue
Cơ chế sử dụng	Queue
Metrics	Queue Length, Latency
Expected Result	Không mất dữ liệu
Pass Criteria	Queue hoạt động ổn định

Dữ liệu runtime được sử dụng để theo dõi độ dài hàng đợi, độ trễ truyền dữ liệu và khả năng xử lý các sự kiện đồng thời của hệ thống.

5.8.4 CON-01 — Drone Xử Lý Nhiều Cảm Biến

Trong kịch bản này, drone đồng thời nhận dữ liệu từ nhiều cảm biến khoảng cách và cảm biến chuyển động. Nhiều task cùng hoạt động song song nhằm xử lý và hợp nhất dữ liệu trước khi đưa ra quyết định điều khiển.

Mục tiêu của test case là đánh giá khả năng xử lý concurrency của scheduler khi nhiều task ở trạng thái Ready cùng lúc.

Bảng 5.21. CON-01 Configuration

Thuộc tính	Giá trị
Test Suite	Concurrency Testing
Kịch bản	Drone Xử Lý Nhiều Cảm Biến
Task tham gia	Sensor_Task, Control_Task, Logger_Task
Metrics	CPU Usage, Context Switch Count
Expected Result	Hệ thống hoạt động ổn định
Pass Criteria	Không xuất hiện starvation

5.8.5 CON-02 — Nhà Máy Nhiều Tín Hiệu Điều Khiển

Test case này mô phỏng môi trường sản xuất với nhiều tín hiệu điều khiển được tạo ra đồng thời. Các task phải chia sẻ tài nguyên và phối hợp với nhau để duy trì hoạt động ổn định của hệ thống.

Bảng 5.22. CON-02 Configuration

Thuộc tính	Giá trị
Test Suite	Concurrency Testing
Kịch bản	Nhà Máy Nhiều Tín Hiệu Điều Khiển
Shared Resource	System Status Variable
Metrics	Queue Delay, Response Time
Expected Result	Không xảy ra deadlock
Pass Criteria	Hệ thống tiếp tục hoạt động

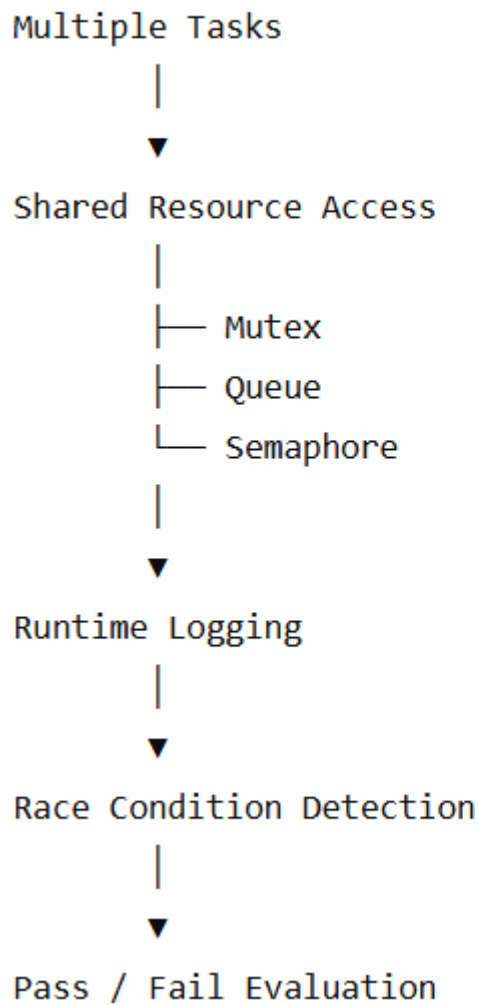
Để hỗ trợ phân tích synchronization, hệ thống ghi nhận các sự kiện lock, unlock, queue send và queue receive trong quá trình thực thi.

```
log_event(  
    "Mutex",  
    "LOCK_ACQUIRED",  
    xTaskGetTickCount()  
);
```

Đoạn mã trên minh họa cách hệ thống ghi nhận các sự kiện synchronization trong quá trình kiểm thử.

Hình 5.8. Synchronization và Concurrency Testing Workflow

Hình 5.8. Synchronization và Concurrency Testing Workflow



Thông qua nhóm kiểm thử Synchronization và Concurrency, đề tài đánh giá khả năng phối hợp giữa các task trong môi trường đa nhiệm, đồng thời xác minh hiệu quả của các cơ chế mutex, queue và semaphore trong việc bảo vệ tài nguyên dùng chung và duy trì tính ổn định của hệ thống.

5.9 Fault Injection Testing

Bên cạnh các điều kiện hoạt động bình thường, đề tài còn xây dựng Fault Injection Testing nhằm đánh giá mức độ ổn định của hệ thống khi xuất hiện các lỗi mô phỏng có chủ đích.

Các lỗi được tạo bởi Fault_Injector_Task bao gồm CPU overload, queue congestion, artificial delay và invalid sensor data. Kết quả thu được được sử dụng để đánh giá khả năng duy trì hoạt động của scheduler, synchronization mechanism và deadline handling.

Trong hệ thống FreeRTOS Simulator của đề tài, Fault_Injector_Task được sử dụng để tạo các lỗi mô phỏng có kiểm soát nhằm kiểm tra khả năng thích ứng của scheduler và các task chức năng. Nhóm kiểm thử Fault Injection Testing tập trung đánh giá hành vi của hệ thống khi xuất hiện lỗi runtime và phân tích tác động của lỗi tới response time, deadline, synchronization và timing metrics.

Bảng 5.24. Fault Injection Testing Summary

Mã Test Case	Kịch bản kiểm thử	Fault Type	Metrics
FLT-01	CPU Overload Trong Nhà Máy	CPU Load Injection	CPU Usage, Deadline Miss
FLT-02	Queue Blocking Khi Truyền Dữ Liệu	Queue Congestion	Queue Length, Latency
FLT-03	Artificial Delay Trong Drone Control	Task Delay Injection	Response Time, WCET
FLT-04	Sensor Nhiễu Dữ Liệu	Invalid Input Injection	Execution Time, Error Detection

5.9.1 FLT-01 — CPU Overload Trong Nhà Máy

Kịch bản này mô phỏng môi trường sản xuất công nghiệp trong đó nhiều task đồng thời yêu cầu xử lý CPU với tải cao hơn điều kiện vận hành bình thường.

Mục tiêu của test case là đánh giá khả năng hoạt động của scheduler khi hệ thống chịu áp lực CPU lớn. Fault_Injector_Task được sử dụng để sinh thêm workload nhằm tăng mức sử dụng CPU và tạo tình huống CPU overload.

Bảng 5.25. FLT-01 Configuration

Thuộc tính	Giá trị
Test Suite	Fault Injection Testing
Kịch bản	CPU Overload Trong Nhà Máy
Fault Type	CPU Load Injection
Task tham gia	Control_Task, Logger_Task, Fault_Injector_Task
Metrics	CPU Usage, Deadline Miss Count
Expected Result	Hệ thống vẫn duy trì hoạt động
Pass Criteria	Không crash và deadline miss trong ngưỡng cho phép

Trong quá trình kiểm thử, hệ thống theo dõi mức sử dụng CPU, số lượng deadline miss và khả năng duy trì hoạt động của scheduler dưới điều kiện tải cao.

5.9.2 FLT-02 — Queue Blocking Khi Truyền Dữ Liệu

Test case này mô phỏng hiện tượng queue congestion trong quá trình truyền dữ liệu giữa các task.

Fault_Injector_Task được sử dụng để làm chậm tốc độ xử lý queue hoặc tạo thêm dữ liệu đầu vào nhằm gây tắc nghẽn hàng đợi. Mục tiêu của test case là đánh giá ảnh hưởng của queue blocking tới latency, response time và khả năng trao đổi dữ liệu của hệ thống.

Bảng 5.26. FLT-02 Configuration

Thuộc tính	Giá trị
Test Suite	Fault Injection Testing
Kịch bản	Queue Blocking Khi Truyền Dữ Liệu
Fault Type	Queue Congestion
Shared Resource	Communication Queue
Metrics	Queue Length, Latency
Expected Result	Queue vẫn xử lý dữ liệu
Pass Criteria	Không mất dữ liệu nghiêm trọng

Trong quá trình thực hiện, hệ thống ghi nhận độ dài queue, thời gian chờ xử lý và mức độ ảnh hưởng của congestion tới các task liên quan.

5.9.3 FLT-03 — Artificial Delay Trong Drone Control

Kịch bản này mô phỏng việc xuất hiện delay bất thường trong Control_Task của hệ thống drone tránh chướng ngại vật.

Fault_Injector_Task được cấu hình để chèn delay nhân tạo vào task điều khiển nhằm tạo điều kiện phân tích tác động của thời gian thực thi kéo dài tới deadline và response time.

Bảng 5.27. FLT-03 Configuration

Thuộc tính	Giá trị
Test Suite	Fault Injection Testing
Kịch bản	Artificial Delay Trong Drone Control
Fault Type	Task Delay Injection
Task tham gia	Control_Task, Sensor_Task, Fault_Injector_Task

Metrics	Response Time, WCET
Expected Result	Scheduler vẫn phản ứng đúng
Pass Criteria	Response Time nằm trong threshold thiết kế

Test case này cho phép đánh giá độ nhạy của hệ thống trước các trường hợp task thực thi lâu hơn bình thường và khả năng duy trì deadline dưới tác động của delay.

5.9.4 FLT-04 — Sensor Nhiều Dữ Liệu

Test case này mô phỏng việc cảm biến tạo dữ liệu sai lệch hoặc dữ liệu nhiễu trong quá trình vận hành hệ thống.

Fault_Injector_Task sinh dữ liệu bất thường và truyền tới Sensor_Task nhằm đánh giá khả năng phát hiện lỗi và xử lý dữ liệu không hợp lệ của hệ thống.

Bảng 5.28. FLT-04 Configuration

Thuộc tính	Giá trị
Test Suite	Fault Injection Testing
Kịch bản	Sensor Nhiều Dữ Liệu
Fault Type	Invalid Input Injection
Task tham gia	Sensor_Task, Control_Task
Metrics	Execution Time, Error Detection
Expected Result	Hệ thống phát hiện dữ liệu lỗi
Pass Criteria	Không gây crash hệ thống

Thông qua test case này, đề tài đánh giá khả năng duy trì ổn định của hệ thống trước các dữ liệu đầu vào bất thường.

Để hỗ trợ fault injection, hệ thống sử dụng các hàm mô phỏng lỗi trong quá trình chạy test case:

```

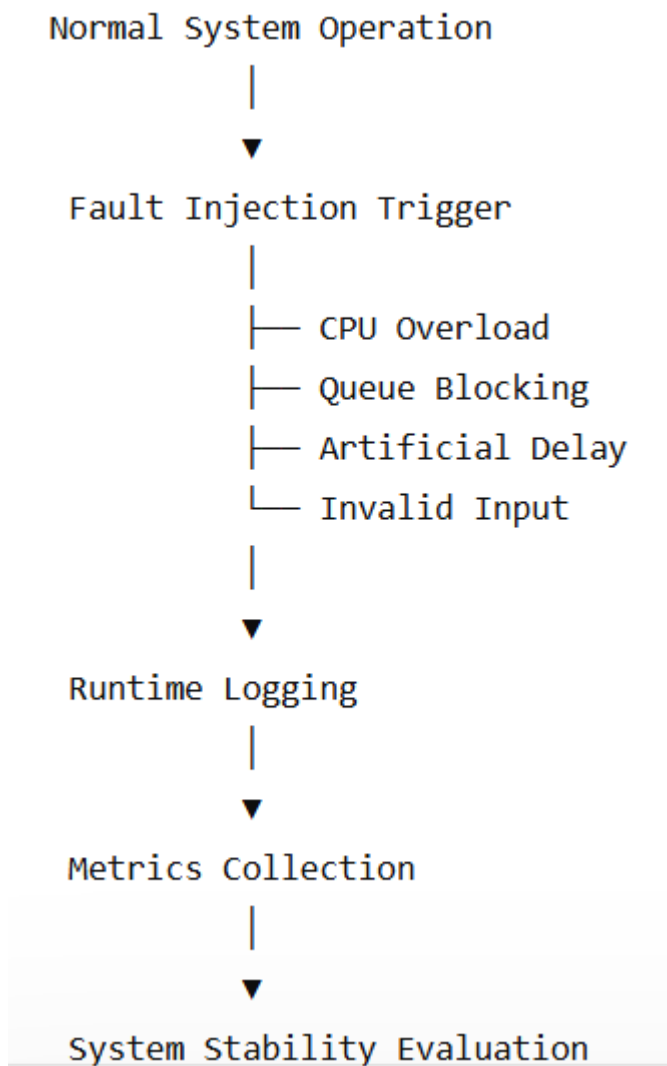
fault_inject_cpu_overload();

log_event(
    "Fault_Injector_Task",
    "CPU_OVERLOAD",
    xTaskGetTickCount()
);

```


Đoạn mã trên minh họa cách Fault_Injector_Task tạo tải CPU nhân tạo và ghi nhận sự kiện fault injection vào runtime log.

Hình 5.9. Fault Injection Testing Workflow



Thông qua nhóm kiểm thử Fault Injection, đề tài đánh giá mức độ ổn định của hệ thống trong các điều kiện lỗi có kiểm soát, đồng thời phân tích khả năng phản ứng của scheduler, synchronization mechanism và deadline handling khi môi trường vận hành không còn lý tưởng.

5.10 Timing Analysis Testing

Sau khi hoàn thành các nhóm kiểm thử chức năng, đề tài thực hiện Timing Analysis Testing nhằm thu thập và phân tích các chỉ số thời gian thực của hệ thống.

Các metrics như Execution Time, Response Time, Latency, WCET, CPU Usage, Context Switch Count và Deadline Miss Count được sử dụng để đánh giá hiệu năng của

FreeRTOS Simulator và làm cơ sở cho hoạt động phân tích kết quả ở Chương 6. Trong đề tài này, Timing Analysis được thực hiện dựa trên dữ liệu runtime thu thập từ FreeRTOS Simulator. Các metrics được sử dụng bao gồm Execution Time, Response Time, Latency, Worst-Case Execution Time (WCET), CPU Usage, Context Switch Count và Deadline Miss Count. Những chỉ số này cho phép đánh giá hành vi của scheduler, hiệu quả của priority scheduling và mức độ đáp ứng deadline của các task trong môi trường đa nhiệm.

Bảng 5.29. Timing Analysis Testing Summary

Mã Test Case	Kịch bản kiểm thử	Metrics chính
TIM-01	Phân Tích Response Time của Emergency_Task	Response Time
TIM-02	Phân Tích WCET của Control_Task	WCET, Execution Time
TIM-03	Phân Tích Context Switch khi Preemption	Context Switch Count
TIM-04	Phân Tích CPU Usage dưới Tải Cao	CPU Usage, Latency

5.10.1 TIM-01 — Phân Tích Response Time của Emergency_Task

Test case này được xây dựng nhằm đánh giá khả năng phản hồi của hệ thống khi xuất hiện các sự kiện khẩn cấp.

Trong kịch bản kiểm thử, Emergency_Task được kích hoạt bởi một sự kiện mô phỏng như phát hiện cháy hoặc tín hiệu Emergency Stop. Hệ thống ghi nhận thời điểm task chuyển sang trạng thái Ready và thời điểm task thực sự bắt đầu thực thi.

Bảng 5.30. TIM-01 Configuration

Thuộc tính	Giá trị
Test Suite	Timing Analysis
Kịch bản	Emergency Event Handling
Task tham gia	Emergency_Task
Metrics	Response Time
Expected Result	Response Time nhỏ hơn deadline
Pass Criteria	Response Time $\leq$ 20 ms

Mục tiêu của test case là xác định khả năng phản ứng của scheduler đối với các nhiệm vụ có mức ưu tiên cao nhất trong hệ thống.

5.10.2 TIM-02 — Phân Tích WCET của Control_Task

Kịch bản này tập trung đánh giá Worst-Case Execution Time (WCET) của Control_Task trong điều kiện hệ thống hoạt động với nhiều task đồng thời.

WCET được sử dụng để xác định thời gian thực thi lớn nhất mà Control_Task có thể gặp phải trong quá trình vận hành. Đây là thông số quan trọng trong việc đánh giá khả năng đáp ứng deadline của hệ thống thời gian thực.

Bảng 5.31. TIM-02 Configuration

Thuộc tính	Giá trị
Test Suite	Timing Analysis
Kịch bản	Control Processing
Task tham gia	Control_Task
Metrics	WCET, Execution Time
Expected Result	WCET nhỏ hơn deadline
Pass Criteria	$WCET \leq \text{Deadline}$

Thông qua test case này, nhóm đánh giá mức độ an toàn của thiết kế thời gian thực và khả năng vận hành của task trong điều kiện tải cao.

5.10.3 TIM-03 — Phân Tích Context Switch khi Preemption

Test case này tập trung đánh giá hành vi của scheduler khi xảy ra preemption.

Trong quá trình kiểm thử, Emergency_Task được kích hoạt trong lúc một task khác đang chạy. Scheduler sẽ thực hiện context switch để chuyển CPU sang Emergency_Task.

Bảng 5.32. TIM-03 Configuration

Thuộc tính	Giá trị
Test Suite	Timing Analysis
Kịch bản	Priority Preemption
Task tham gia	Emergency_Task, Control_Task
Metrics	Context Switch Count
Expected Result	Context switch diễn ra đúng
Pass Criteria	Scheduler thực hiện preemption thành công

Dữ liệu context switch được sử dụng để đánh giá hiệu quả của cơ chế priority scheduling và priority preemption trong FreeRTOS.

5.10.4 TIM-04 — Phân Tích CPU Usage dưới Tải Cao

Kịch bản này mô phỏng hệ thống hoạt động trong điều kiện tải cao với nhiều task đồng thời thực thi.

Mục tiêu của test case là đánh giá mức sử dụng CPU và ảnh hưởng của tải hệ thống tới latency và response time.

Bảng 5.33. TIM-04 Configuration

Thuộc tính	Giá trị
Test Suite	Timing Analysis
Kịch bản	High CPU Load
Task tham gia	Toàn bộ task
Metrics	CPU Usage, Latency
Expected Result	Hệ thống vẫn ổn định
Pass Criteria	CPU Usage trong ngưỡng thiết kế

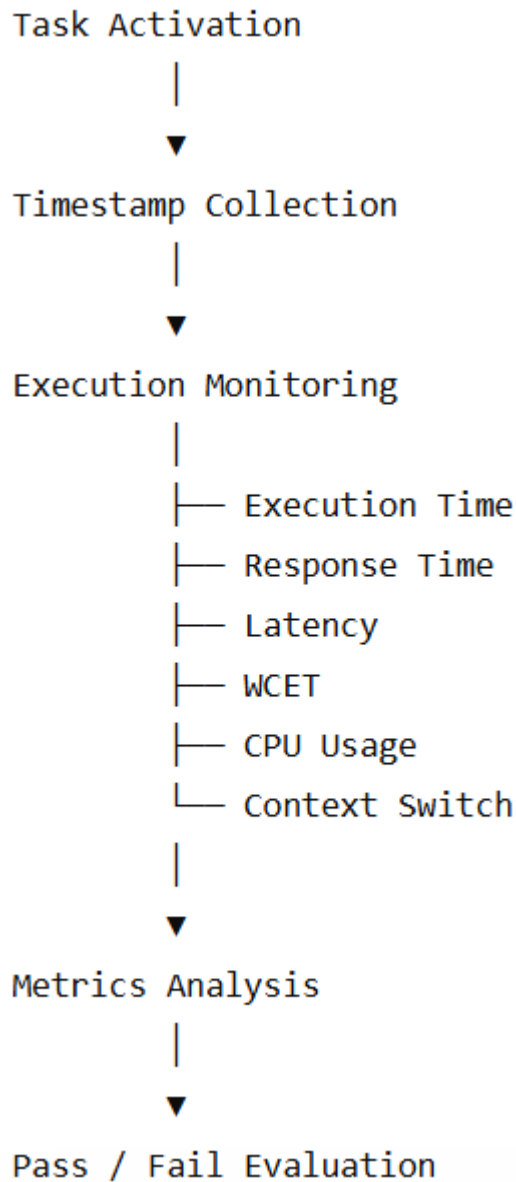
Test case này giúp xác định giới hạn vận hành của hệ thống và khả năng duy trì hiệu năng khi số lượng task hoặc workload tăng lên.

Để hỗ trợ Timing Analysis, hệ thống sử dụng cơ chế ghi nhận timestamp tại các thời điểm quan trọng trong vòng đời của task:

```
uint32_t start_time = xTaskGetTickCount();  
  
/* Task Processing */  
  
uint32_t end_time = xTaskGetTickCount();  
  
execution_time = end_time - start_time;
```

Đoạn mã trên minh họa cách hệ thống đo thời gian thực thi của task trong quá trình thu thập metrics.

Hình 5.10. Timing Analysis Workflow



Thông qua nhóm kiểm thử Timing Analysis, đề tài xây dựng cơ sở định lượng để đánh giá hiệu năng của hệ thống FreeRTOS Simulator. Các metrics thu được từ nhóm kiểm thử này sẽ được sử dụng trực tiếp trong Chương 6 để phân tích kết quả thực nghiệm, so sánh expected result với actual result và đánh giá mức độ đáp ứng thời gian thực của hệ thống.

5.11 Tổng hợp Test Suite

Sau khi xây dựng các nhóm kiểm thử cho scheduler, priority scheduling, priority preemption, deadline handling, synchronization, concurrency, fault injection và timing analysis, toàn bộ test case được tổng hợp thành một bộ kiểm thử thống nhất nhằm phục vụ quá trình thực nghiệm và đánh giá hệ thống.

Việc tổ chức test case thành các test suite giúp quá trình kiểm thử được thực hiện có hệ thống, đồng thời đảm bảo tất cả các cơ chế quan trọng của FreeRTOS Simulator đều được đánh giá thông qua các kịch bản thực tế đã xây dựng trong Chương 1. Mỗi test suite tập trung vào một nhóm chức năng cụ thể nhưng vẫn có sự liên kết với các nhóm còn lại thông qua hệ thống metrics và cơ chế pass/fail đã trình bày ở các mục trước.

Bảng 5.34. Tổng hợp Test Suite

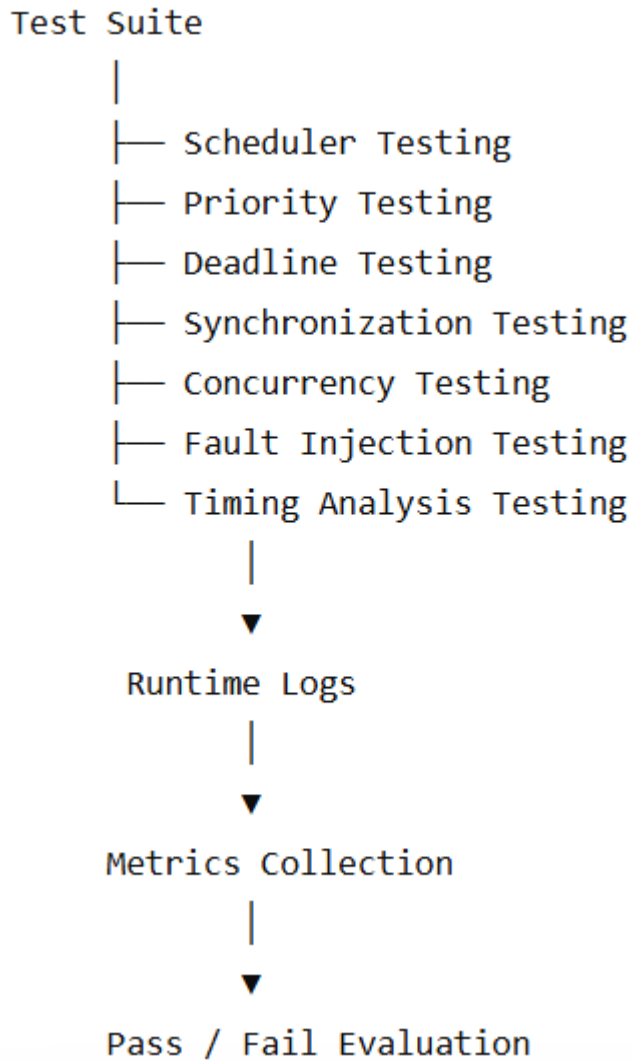
Test Suite	Số Test Case	Trọng tâm đánh giá
Scheduler Testing	3	Execution Order, Task State
Priority Scheduling Testing	2	Priority Handling
Priority Preemption Testing	1	Preemption Response
Hard Deadline Testing	2	Deadline Miss, WCET
Soft Deadline Testing	2	Latency, Response Time
Synchronization Testing	3	Mutex, Queue, Semaphore
Concurrency Testing	3	Multi-Task Execution
Fault Injection Testing	4	System Robustness
Timing Analysis Testing	4	Runtime Metrics

Bảng 5.35. Tổng số Test Case

Hạng mục	Số lượng
Tổng Test Suite	9
Tổng Test Case	24
Runtime Metrics	9
Kịch bản thực tế	10+

Thông qua bộ test suite này, đề tài có thể đánh giá toàn diện hành vi của hệ thống từ mức độ hoạt động của scheduler, hiệu quả priority scheduling, khả năng đáp ứng deadline cho tới tính ổn định khi xuất hiện lỗi mô phỏng. Đồng thời, việc sử dụng cùng một tập metrics cho tất cả các nhóm kiểm thử giúp kết quả thực nghiệm có tính nhất quán và thuận lợi cho việc phân tích ở chương tiếp theo.

Hình 5.11. Cấu trúc bộ kiểm thử tổng thể



5.12 Tổng kết chương

Chương 5 đã trình bày quá trình thiết kế bộ kiểm thử cho hệ thống nhúng đa tiến trình thời gian thực sử dụng FreeRTOS Simulator. Trên cơ sở các yêu cầu đã xác định trong Chương 1, nền tảng lý thuyết ở Chương 2, kiến trúc hệ thống ở Chương 3 và quá trình cài đặt ở Chương 4, nhóm đã xây dựng chiến lược kiểm thử, môi trường kiểm thử, hệ thống runtime metrics và tiêu chí pass/fail phục vụ cho hoạt động đánh giá hệ thống.

Các test suite được tổ chức theo các nhóm chức năng chính bao gồm Scheduler Testing, Priority Scheduling, Priority Preemption, Hard Deadline Testing, Soft Deadline Testing, Synchronization Testing, Concurrency Testing, Fault Injection Testing và Timing Analysis Testing. Mỗi nhóm kiểm thử đều được xây dựng dựa trên các kịch bản thực tế nhằm đánh giá khả năng đáp ứng của scheduler, hiệu quả đồng bộ hóa, mức độ tuân thủ deadline và tính ổn định của hệ thống trong nhiều điều kiện vận hành khác nhau.

Bên cạnh đó, chương này cũng xác định các metrics quan trọng như Execution Time, Response Time, Latency, WCET, CPU Usage, Context Switch Count và Deadline Miss Count. Những chỉ số này đóng vai trò là cơ sở định lượng cho việc đánh giá hiệu năng và mức độ đáp ứng thời gian thực của hệ thống.

Trên cơ sở bộ kiểm thử đã được thiết kế, Chương 6 sẽ trình bày quá trình thực hiện các test case, thu thập dữ liệu runtime, phân tích kết quả thực nghiệm và đánh giá mức độ đáp ứng của hệ thống FreeRTOS Simulator thông qua các metrics và tiêu chí pass/fail đã xác định.

CHƯƠNG 6: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

6.1 Môi trường và kịch bản thực nghiệm

Chương này sử dụng hai lần chạy của hệ thống Embedded_RTOS_Testing Simulator: chạy bình thường (Baseline Run) và chạy có lỗi (Fault Injection Run). Dữ liệu runtime log được thu thập và phân tích nhằm đánh giá cơ chế lập lịch, ưu tiên tác vụ, giám sát deadline, đồng bộ hóa, xử lý đồng thời và các chỉ số thời gian.

6.1.1 Môi trường thực nghiệm

Hệ thống mô phỏng các cơ chế đặc trưng của RTOS trên nền PC:

Thành phần	Giá trị
Platform	Embedded_RTOS_Testing Simulator
Ngôn ngữ lập trình	C
Scheduler	Priority-Based Preemptive
Runtime Logging	Enabled
Fault Injection	Enabled
Metrics Collected	Response Time, Execution Time, WCET, Deadline Miss, Context Switch, CPU Usage
Output	CSV log + Charts
Analysis Tool	Python Log Analyzer

Các task được cấu hình theo priority, period, WCET và deadline. Runtime log ghi nhận trạng thái task, sự kiện lỗi và các hoạt động đồng bộ hóa.

6.1.2 Kịch bản thực nghiệm

Để đánh giá toàn diện, nhóm xây dựng hai kịch bản kiểm thử:

Scenario	Mục tiêu	Điều kiện
Baseline Run	Kiểm tra hoạt động bình thường	Không chèn delay nhân tạo
Fault Injection Run	Đánh giá khả năng phát hiện lỗi thời gian thực	Artificial delay cho Control_Task, mô phỏng CPU overload

Các kịch bản thực nghiệm được xây dựng từ các test case ở Chương 5 nhằm mô phỏng các tình huống thực tế của hệ thống những thời gian thực:

- Camera an ninh phát hiện xâm nhập.
- Nhà máy có chức năng dừng khẩn cấp (Emergency Stop).
- Hệ thống báo cháy thông minh.
- Trạm quan trắc thời tiết.
- Kịch bản chen lỗi (Fault Injection) để tạo deadline miss có kiểm soát.

6.1.3 Quy trình thực nghiệm

Quá trình thực nghiệm gồm ba bước:

1. Baseline Run

- Biên dịch và chạy hệ thống với cấu hình bình thường.
- Thu thập runtime log và các chỉ số đánh giá.

2. Fault Injection Run

- Tăng thời gian thực thi của Control_Task để mô phỏng quá tải.
- Thu thập runtime log và các chỉ số đánh giá.

3. Fault Injection Run

- Xử lý log bằng Python Log Analyzer.
- So sánh kết quả giữa hai lần chạy để đánh giá cơ chế lập lịch, giám sát deadline và hiệu năng hệ thống.

6.2. Test Execution Matrix

Các test case được thực thi trên hệ thống Embedded_RTOS_Testing Simulator và kết quả được tổng hợp trong Bảng 6.2.

Bảng 6.2. Kết quả thực hiện Test Case

TC ID	Test Case	Scenario	Runtime Evidence	Result
TC01	Scheduler Testing	Sensor Kiểm Soát Người Ra Vào	CONTEXT_SWITCH	PASS
TC02	Priority Scheduling Testing	Nhà Máy Emergency Stop	READY → START Order	PASS
TC03	Priority Preemption Testing	Camera An Ninh Phát Hiện Xâm Nhập	PREEMPTED Event	PASS
TC04	Hard Deadline Testing	Hệ Thống Báo Cháy Thông Minh	Deadline Miss = 0	PASS
TC05	Soft Deadline Testing	Trạm Quan Trắc Thời Tiết	Response Time hợp lệ	PASS
TC06	Synchronization Testing	Queue + Mutex Communication	QUEUE_SEND / MUTEX_LOCK	PASS
TC07	Concurrency Testing	Drone Tránh Chướng Ngại Vật	Runtime Stable	PASS
TC08	Fault Injection Testing	CPU Overload Scenario	DEADLINE_MISS	FAIL
TC09	Timing Analysis Testing	Runtime Metrics Analysis	WCET / Response Time	PASS

6.3 Kịch bản Camera An Ninh Phát Hiện Xâm Nhập

Kịch bản Camera An Ninh Phát Hiện Xâm Nhập được xây dựng nhằm kiểm tra cơ chế Priority Scheduling và Priority Preemption của hệ thống. Kịch bản này mô phỏng tình huống camera phát hiện đối tượng xâm nhập trong khi các task khác đang hoạt động.

Runtime Log

Tại thời điểm 220 ms, Sensor_Task phát hiện sự kiện xâm nhập và kích hoạt Emergency_Task.

220,Sensor_Task,INTRUSION_DETECTED

220,Emergency_Task,READY

220,Deadline_Monitor_Task,PREEMPTED

220,Emergency_Task,START

229,Emergency_Task,FINISH

Log cho thấy Deadline_Monitor_Task đang thực thi bị preempt ngay khi Emergency_Task xuất hiện. Scheduler lập tức chuyển CPU cho Emergency_Task.

Kết quả thực nghiệm

Metric	Giá trị
Average Response Time	0 ms
Maximum Response Time	0 ms
WCET	9 ms
Deadline	20 ms
Deadline Miss	0

Đánh giá

Kết quả cho thấy Emergency_Task được thực thi ngay khi xuất hiện và hoàn thành trước deadline. Cơ chế Priority Preemption hoạt động đúng thiết kế. Test case PASS.

6.4 Kịch bản Nhà Máy Có Emergency Stop

Kịch bản Nhà Máy Có Emergency Stop được xây dựng nhằm đánh giá tính ổn định của scheduler khi xuất hiện tác vụ khẩn cấp trong môi trường đa nhiệm. Kịch bản tập trung kiểm tra khả năng duy trì hoạt động ổn định của hệ thống sau khi xảy ra sự kiện khẩn cấp.

Mục tiêu kiểm thử

- Đánh giá tính ổn định của scheduler khi xuất hiện tác vụ khẩn cấp.
- Kiểm tra khả năng phục hồi và duy trì hoạt động của hệ thống.
- Đánh giá hiện tượng starvation và context switch trong quá trình xử lý.

Các task tham gia

Task	Priority
Emergency_Task	5
Control_Task	4
Sensor_Task	3
Communication_Task	2

Kết quả thực nghiệm

Emergency_Task được kích hoạt để mô phỏng tình huống dừng khẩn cấp. Scheduler ưu tiên xử lý tác vụ này, sau đó tiếp tục thực thi các task còn lại theo đúng mức ưu tiên. Runtime log không ghi nhận hiện tượng treo hoặc mất phản hồi của hệ thống.

Bảng 6.5. Kết quả đánh giá Scheduler Stability

Metric	Giá trị
Scheduler Result	PASS
Starvation Detected	No
Deadlock Detected	No
Deadline Miss	0
System Recovery	PASS

Đánh giá

Kịch bản này tập trung đánh giá tính ổn định của scheduler khi xuất hiện tác vụ khẩn cấp. Kết quả thực nghiệm cho thấy khi Emergency_Task được kích hoạt, scheduler ưu tiên cấp CPU để xử lý sự cố, sau đó các task còn lại tiếp tục thực thi bình thường theo thiết kế. Không ghi nhận hiện tượng starvation, deadlock hoặc mất khả năng phản hồi. Kết quả cho thấy scheduler duy trì được tính ổn định của hệ thống và test case được đánh giá PASS.

6.5 Kịch bản Hệ Thống Báo Cháy Thông Minh

Kịch bản Hệ Thống Báo Cháy Thông Minh được sử dụng để đánh giá khả năng đáp ứng hard deadline và cơ chế giám sát deadline của hệ thống.

Mục tiêu kiểm thử

- Đánh giá khả năng đáp ứng hard deadline của hệ thống.
- Kiểm tra khả năng phát hiện deadline miss.
- Thu thập các timing metrics phục vụ phân tích thời gian thực.

Các task tham gia

Task	Deadline
Emergency_Task	20 ms
Control_Task	50 ms
Sensor_Task	100 ms

Kết quả Baseline Run

Task	WCET	Deadline	Deadline Miss
Emergency Task	9 ms	20 ms	0
Control Task	22 ms	50 ms	0
Sensor Task	9 ms	100 ms	0

Bảng 6.6. Kết quả Hard Deadline Testing

Tiêu chí đánh giá	Kết quả
Emergency Task đáp ứng deadline	PASS
Control Task đáp ứng deadline	PASS
Sensor Task đáp ứng deadline	PASS
Deadline Miss Count	0
Hard Deadline Testing	PASS

Kết quả thực nghiệm cho thấy tất cả các task đều hoàn thành trước deadline và không ghi nhận deadline miss. Kết quả Baseline Run được sử dụng làm giá trị tham chiếu cho Fault Injection Run ở Mục 6.7.

Đánh giá

Kết quả cho thấy cơ chế Deadline Monitoring hoạt động đúng thiết kế. Tất cả các task đều đáp ứng deadline, không ghi nhận deadline miss và test case được đánh giá PASS.

6.6 Kịch bản Trạm Quan Trắc Thời Tiết

Kịch bản Trạm Quan Trắc Thời Tiết được sử dụng để đánh giá khả năng đáp ứng soft deadline, trao đổi dữ liệu giữa các task và hiệu quả của các cơ chế synchronization trong hệ thống.

Mục tiêu kiểm thử

- Đánh giá khả năng đáp ứng soft deadline.
- Đánh giá cơ chế Queue Communication và Mutex Synchronization.
- Kiểm tra sự ổn định của hệ thống khi nhiều task cùng truy cập tài nguyên dùng chung.

Các task tham gia

Task	Chức năng
Sensor Task	Thu thập dữ liệu môi trường
Communication Task	Truyền dữ liệu tới hệ thống giám sát
Logger Task	Ghi nhận sự kiện runtime

Kết quả thực nghiệm

Runtime log ghi nhận dữ liệu được truyền thành công giữa các task thông qua queue mà không xuất hiện mất dữ liệu hoặc lỗi truyền thông. Bên cạnh đó, Logger_Task sử dụng mutex để bảo vệ vùng nhớ dùng chung trong quá trình ghi log. Log cho thấy mutex được khóa trước khi ghi dữ liệu và giải phóng sau khi hoàn thành, giúp ngăn chặn truy cập đồng thời gây sai lệch dữ liệu.

Bảng 6.7. Kết quả Synchronization Testing

Kiểm thử	Kết quả
Queue Communication	PASS
Mutex Synchronization	PASS
Race Condition	Not Detected
Deadlock	Not Detected
Priority Inversion	Not Detected
Soft Deadline Compliance	PASS

Đánh giá

Không ghi nhận hiện tượng race condition, deadlock hoặc priority inversion trong quá trình thực nghiệm. Queue communication và mutex synchronization hoạt động hiệu quả, đảm bảo tính nhất quán dữ liệu và sự ổn định của hệ thống. Kết quả cho thấy các cơ chế synchronization hoạt động đúng theo thiết kế và test case được đánh giá PASS.

6.7 Kịch bản Fault Injection

Kịch bản Fault Injection được sử dụng để đánh giá khả năng phát hiện lỗi thời gian thực của hệ thống bằng cách tạo artificial delay cho Control_Task, từ đó gây ra deadline violation có kiểm soát.

Mục tiêu kiểm thử

- Đánh giá khả năng phát hiện deadline miss.
- Kiểm tra tính ổn định của scheduler khi xuất hiện lỗi.
- Đánh giá sự thay đổi của timing metrics dưới điều kiện tải cao.

Các task tham gia

Task	Chức năng
Control_Task	Xử lý điều khiển
Fault_Injector_Task	Sinh lỗi kiểm thử
Deadline_Monitor_Task	Giám sát deadline
Logger_Task	Ghi nhận runtime log

Trong Baseline Run, Fault_Injector_Task chỉ tạo mức tải thấp và không gây ảnh hưởng đáng kể đến Control_Task. Trong Fault Injection Run, artificial delay được tăng lên nhằm làm tăng execution time của Control_Task vượt quá deadline thiết kế.

Bảng 6.6. So sánh Baseline Run và Fault Injection Run

Metric	Baseline Run	Fault Injection Run
Control_Task WCET	22 ms	59 ms
Control_Task Deadline	50 ms	50 ms
Deadline Miss Count	0	1
Result	PASS	FAIL

Khi artificial delay được kích hoạt, WCET của Control_Task tăng từ 22 ms lên 59 ms và vượt quá deadline 50 ms. Runtime log ghi nhận sự kiện DEADLINE_MISS, được Deadline_Monitor_Task phát hiện và ghi nhận, cho thấy cơ chế giám sát deadline hoạt động đúng thiết kế.

Đánh giá

Fault Injection Run đã tạo ra deadline miss có kiểm soát. Mặc dù xuất hiện lỗi, scheduler vẫn duy trì hoạt động ổn định và các task còn lại hoàn thành trong thời gian cho phép. Kết quả cho thấy hệ thống có khả năng phát hiện và ghi nhận lỗi thời gian thực.

6.8 Đánh giá Timing Analysis

Timing Analysis được thực hiện dựa trên các runtime metrics thu thập từ hệ thống, bao gồm Response Time, Execution Time, Worst-Case Execution Time (WCET) và Deadline Miss Count. Các chỉ số này được sử dụng để đánh giá khả năng đáp ứng thời gian thực của từng task cũng như hiệu quả hoạt động của scheduler trong cả điều kiện bình thường và điều kiện xuất hiện lỗi.

Bảng 6.9. Kết quả Timing Metrics (Baseline Run)

Task	Avg Response Time (ms)	Max Response Time (ms)	WCET (ms)
Emergency_Task	0.00	0	9
Control_Task	0.42	10	22
Deadline_Monitor_Task	15.83	25	19
Sensor_Task	26.67	35	9
Communication_Task	26.00	35	14
Fault_Injector_Task	48.33	65	49
Logger_Task	58.33	85	54

Kết quả cho thấy Emergency_Task có Response Time bằng 0 ms, phản ánh khả năng phản ứng tức thời của scheduler đối với task ưu tiên cao. Control_Task có Average Response Time 0.42 ms và WCET 22 ms, thấp hơn deadline 50 ms, cho thấy các tác vụ điều khiển được xử lý hiệu quả. Mặc dù Logger_Task có Response Time cao nhất do mức ưu tiên thấp, task vẫn hoàn thành trong giới hạn deadline cho phép.

Bảng 6.10. So sánh Timing Metrics giữa Baseline Run và Fault Injection Run

Metric	Baseline Run	Fault Injection Run
Control_Task Avg Response Time	0.42 ms	0.63 ms
Control_Task Max Response Time	10 ms	10 ms
Control_Task WCET	22 ms	59 ms
Control_Task Deadline	50 ms	50 ms
Deadline Miss Count	0	1

Kết quả cho thấy Fault Injection làm WCET của Control_Task tăng từ 22 ms lên 59 ms, vượt deadline 50 ms và gây ra một lần deadline miss. Mặc dù Average Response Time chỉ tăng nhẹ, sự gia tăng WCET cho thấy tác động rõ rệt của artificial delay đến khả năng đáp ứng thời gian thực, đồng thời dẫn đến kết quả FAIL của Hard Deadline Testing. Biểu đồ Response Time Distribution cũng cho thấy các task quan trọng duy trì thời gian phản hồi thấp trong điều kiện bình thường, trong khi Control_Task chịu ảnh hưởng đáng kể khi Fault Injection được kích hoạt.

Đánh giá

Các timing metrics thu được phù hợp với thiết kế scheduler và priority architecture. Trong Baseline Run, tất cả các task đều đáp ứng deadline và không ghi nhận bất thường về timing behavior. Trong Fault Injection Run, hệ thống phát hiện thành công deadline violation thông qua sự gia tăng của WCET và Deadline Miss Count, cho thấy cơ chế Timing Analysis và Deadline Monitoring hoạt động hiệu quả. Ngoài các lỗi được tạo có chủ đích, không ghi nhận hiện tượng starvation, deadlock hoặc response time bất thường.

6.9. Đánh giá tổng hợp Pass/Fail

Dựa trên các tiêu chí pass/fail đã xây dựng ở Chương 5, nhóm tiến hành đánh giá kết quả của từng test suite.

Bảng 6.8. Kết quả đánh giá tổng hợp

Test Suite	Baseline Run	Fault Injection Run
Scheduler Testing	PASS	PASS
Priority Scheduling Testing	PASS	PASS

Priority Preemption Testing	PASS	PASS
Synchronization Testing	PASS	PASS
Concurrency Testing	PASS	PASS
Hard Deadline Testing	PASS	FAIL
Soft Deadline Testing	PASS	PASS
Fault Injection Testing	PASS	FAIL
Timing Analysis Testing	PASS	PASS

Kết quả cho thấy toàn bộ test suite đều đạt yêu cầu trong Baseline Run. Trong Fault Injection Run, hệ thống ghi nhận một deadline miss của Control_Task, khiến Hard Deadline Testing và Fault Injection Testing được đánh giá FAIL theo tiêu chí đã đề ra. Tuy nhiên, đây là lỗi được tạo có chủ đích để đánh giá khả năng phát hiện lỗi của hệ thống, trong khi scheduler vẫn duy trì hoạt động ổn định và các task còn lại không bị ảnh hưởng đáng kể.

Đánh giá chung

Kết quả thực nghiệm chứng minh hệ thống có khả năng:

- Quản lý đa nhiệm dựa trên priority scheduling.
- Thực hiện priority preemption đúng thời điểm.
- Giám sát và phát hiện deadline miss.
- Thu thập và phân tích runtime metrics.
- Duy trì hoạt động ổn định khi xuất hiện fault injection.

6.10. Tổng kết chương

Chương 6 đã trình bày kết quả thực nghiệm của hệ thống Embedded_RTOS_Testing Simulator thông qua các kịch bản kiểm thử và dữ liệu runtime thu thập được. Kết quả cho thấy scheduler hoạt động đúng theo cơ chế Priority-Based Preemptive Scheduling, các task đáp ứng deadline trong điều kiện bình thường và duy trì response time phù hợp. Bên cạnh đó, Fault Injection Run đã chứng minh khả năng phát hiện deadline violation và hiệu quả của cơ chế Deadline Monitoring. Nhìn chung, hệ thống đáp ứng các yêu cầu kiểm thử và phù hợp với thiết kế đã đề ra.

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1.1 Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế, triển khai và kiểm thử, đề tài "Xây dựng và kiểm thử hệ nhúng đa tiến trình thời gian thực" đã hoàn thành các mục tiêu đề ra.

Nhóm đã xây dựng thành công môi trường mô phỏng Embedded_RTOS_Testing Simulator dựa trên FreeRTOS, hỗ trợ đa nhiệm, priority scheduling, priority preemption, synchronization, deadline monitoring, runtime logging và fault injection. Các task mô phỏng được triển khai với mức ưu tiên, deadline và cơ chế giao tiếp phù hợp.

Đề tài đã xây dựng và thực hiện thành công bộ kiểm thử cho scheduler, deadline, synchronization, concurrency, fault injection và timing analysis.

Kết quả thực nghiệm cho thấy scheduler hoạt động đúng theo cơ chế Priority-Based Preemptive Scheduling, các task đáp ứng deadline trong điều kiện bình thường và hệ thống có khả năng phát hiện deadline miss thông qua Fault Injection. Trong Baseline Run, toàn bộ test suite đạt PASS; trong Fault Injection Run, hệ thống ghi nhận một deadline miss của Control_Task khi WCET tăng từ 22 ms lên 59 ms và vượt deadline 50 ms.

Thông qua việc phân tích các runtime metrics như Response Time, Execution Time, WCET và Deadline Miss Count, đề tài đã xác nhận tính đúng đắn và hiệu quả của các cơ chế được triển khai.

7.1.2 Đánh giá mức độ hoàn thành mục tiêu đề tài

Dựa trên các kết quả thực nghiệm trình bày trong Chương 6, có thể nhận thấy đề tài đã hoàn thành các mục tiêu đề ra trong proposal.

Bảng 7.1. Đánh giá mức độ hoàn thành mục tiêu đề tài

Mục tiêu	Mức độ hoàn thành
Xây dựng môi trường mô phỏng RTOS	Hoàn thành
Triển khai đa task	Hoàn thành
Scheduler Testing	Hoàn thành
Priority Scheduling Testing	Hoàn thành
Priority Preemption Testing	Hoàn thành
Synchronization Testing	Hoàn thành
Hard Deadline Testing	Hoàn thành
Soft Deadline Testing	Hoàn thành
Fault Injection Testing	Hoàn thành
Timing Analysis	Hoàn thành

Kết quả đánh giá cho thấy toàn bộ các mục tiêu kỹ thuật đã được thực hiện và kiểm chứng thông qua các test case cụ thể. Các kết quả PASS/FAIL thu được từ quá trình thực nghiệm phù hợp với hành vi mong đợi của hệ thống và đáp ứng yêu cầu của đề tài.

7.2 Hạn chế của đề tài

Mặc dù đã đạt được các mục tiêu đề ra, đề tài vẫn còn một số hạn chế. Hệ thống hiện được triển khai trên môi trường mô phỏng nên chưa phản ánh đầy đủ các yếu tố của phần cứng nhúng thực tế. Số lượng task và kịch bản kiểm thử còn giới hạn, trong khi các hệ thống thực tế thường có mức độ tương tác phức tạp hơn.

Bên cạnh đó, cơ chế Fault Injection mới tập trung vào CPU overload và artificial delay, chưa bao quát các lỗi về bộ nhớ, thiết bị ngoại vi hoặc truyền thông. Việc phân tích timing chủ yếu dựa trên runtime metrics và log analysis, chưa tích hợp các công cụ phân tích thời gian thực chuyên dụng.

7.3 Hướng phát triển

Trong tương lai, đề tài có thể được mở rộng theo các hướng sau:

- Triển khai trên phần cứng nhúng thực tế như STM32 hoặc ESP32 kết hợp FreeRTOS.
- Tích hợp các cơ chế synchronization và thuật toán lập lịch nâng cao.
- Mở rộng Fault Injection để mô phỏng các lỗi về bộ nhớ, truyền thông và thiết bị ngoại vi.
- Xây dựng hệ thống thu thập, trực quan hóa và phân tích dữ liệu runtime theo thời gian thực.
- Bổ sung các công cụ phân tích timing chuyên sâu để đánh giá khả năng đáp ứng thời gian thực.
- Tăng số lượng task và kịch bản kiểm thử nhằm mô phỏng các hệ thống quy mô lớn hơn.
- Xây dựng framework kiểm thử tự động hỗ trợ đánh giá và xác minh hệ thống RTOS.

7.4 Tổng kết chương

Chương 7 đã tổng kết kết quả thực hiện, đánh giá các mục tiêu đạt được, các hạn chế và hướng phát triển của đề tài. Kết quả thực nghiệm cho thấy hệ thống đáp ứng các yêu cầu kiểm thử RTOS và hoạt động đúng thiết kế. Nhìn chung, đề tài đã hoàn thành các mục tiêu đề ra và tạo nền tảng cho các nghiên cứu tiếp theo.